



TheKnife

MANUALE TECNICO

Versione 1.0

- Barlera Marco
- Ciani Flavio Angelo
- Gasparini Lorenzo
- Scolaro Gabriele



Indice

1. Introduzione	2
2. Librerie esterne	3
3. Architettura del sistema	5
3.1 Architettura generale.....	5
3.2 Struttura dei moduli.....	7
4. Progettazione del software	11
4.1 Comunicazione client-server.....	11
4.2 Protocollo Request/Response.....	12
4.3 Gestione dei comandi	13
4.4 Gestione delle sessioni.....	14
5. Progettazione della base di dati	18
5.1 Modello concettuale	18
5.2 Diagramma ER.....	22
5.3 Schema della base di dati.....	26
5.4 Vincoli e integrità dei dati	30
6. Scelte progettuali	33
6.1 Gestione dei dati e persistenza	33
6.2 Gestione degli errori	37
6.3 Gestione della sicurezza.....	41
6.4 Gestione dei servizi di geolocalizzazione	45
6.5 Patterns significativi	51
6.6 Ottimizzazioni delle prestazioni	55
7. Installazione ed esecuzione	57
8. Dataset di prova	60
9. Considerazioni progettuali.....	64
10. Limiti della soluzione sviluppata.....	66
11. Bibliografia e sitografia	70



1. Introduzione

TheKnife è un'applicazione sviluppata come parte del Laboratorio Interdisciplinare B del corso di laurea in Informatica presso l'Università degli Studi dell'Insubria.

Il progetto è scritto interamente in Java 21 Temurin ed è stato implementato e testato nei sistemi operativi Windows 10/11 e macOS.

L'applicazione realizza un sistema per la gestione di ristoranti, utenti e recensioni, fornendo funzionalità differenziate in base al ruolo dell'utente.

Il sistema supporta, in particolare, operazioni come registrazione e autenticazione di utenti, ricerca avanzata e visualizzazione dei ristoranti, gestione delle recensioni e dei ristoranti gestiti e utilizzo di funzionalità basate sulla geolocalizzazione.

L'architettura dell'applicazione è stata organizzata in moduli e strati, separando le responsabilità tra la logica di presentazione, la gestione delle richieste, i servizi applicativi e l'accesso ai dati. Tale organizzazione favorisce la manutenibilità del codice e permette l'estensione delle funzionalità del sistema senza dover modificare indiscriminatamente le diverse componenti dell'applicazione.

Il presente manuale descrive le caratteristiche tecniche del sistema, illustrando l'architettura, la struttura dei moduli, le principali scelte progettuali e le soluzioni adottate per la gestione dei dati, degli errori, della sicurezza e della geolocalizzazione.



2. Librerie esterne

- **JavaFX Controls**

- Modulo: client
- Versione: 21.0.2
- Utilizzo: componenti grafici e controlli dell'interfaccia JavaFX
- Descrizione:
 - Fornisce i componenti visivi di base necessari per costruire l'interfaccia grafica utente (GUI) lato client di un'applicazione.
 - Offre nodi pronti all'uso per l'interazione con l'utente (pulsanti, caselle di testo, *slider*, *checkbox*).
 - Contiene componenti avanzati come tabelle (*TableView*) e alberi (*TreeView*) per visualizzare strutture dati.
 - Fornisce componenti integrati per la visualizzazione dei dati tramite grafici (*charts*).
 - Comprende classi per definire l'aspetto grafico e la personalizzazione tramite CSS dei vari controlli.

- **JavaFX FXML**

- Modulo: client
- Versione: 21.0.2
- Utilizzo: FXML per la definizione delle interfacce grafiche
- Descrizione:
 - FXML è un linguaggio basato su XML che serve a definire in modo dichiarativo l'interfaccia grafica (GUI), separando l'aspetto estetico del front-end dalla logica di programmazione scritta in Java.
 - Garantisce buona manutenibilità, in quanto una modifica della posizione di un elemento grafico non richiede la ricompilazione del codice Java del client.
 - Si abbina al pattern Model-View-Controller (MVC).

- **PostgreSQL JDBC Driver**

- Modulo: server
- Versione: 42.7.3
- Utilizzo: connessione e comunicazione con la base di dati PostgreSQL
- Descrizione:



- Serve ad aprire canali di comunicazione tra un server applicativo scritto in Java e una base di dati PostgreSQL.
 - Implementa l'interfaccia JDBC e permette all'applicazione Java di inviare istruzioni SQL alla base di dati PostgreSQL e di riceverne i risultati.
 - Converte i tipi di dati di PostgreSQL in oggetti comprensibili per Java e viceversa.
 - Gestisce le transazioni supportando il controllo delle modifiche ai dati per garantire l'affidabilità delle informazioni.
-
- **BCrypt (*at.favre.lib*)**
 - Modulo: server
 - Versione: 0.10.2
 - Utilizzo: hashing/verifica delle password
 - Descrizione:
 - Serve ad eseguire l'hashing delle password degli utenti utilizzando l'algoritmo BCrypt, evitando di memorizzare le password in chiaro nella base di dati.
 - Utilizza un valore casuale (*salt*) per impedire attacchi con tabelle precompilate (*rainbow tables*).
 - Rende deliberatamente lento il calcolo dell'hash. Questo impedisce di tentare milioni di combinazioni veloci (*brute force*).



3. Architettura del sistema

3.1 Architettura generale

Il sistema è organizzato in tre moduli:

- ***theknife-common***
 - Contiene gli elementi condivisi tra client e server, in particolare il contratto di comunicazione, i DTO, gli enumerativi e le configurazioni condivise.
- ***theknife-client***
 - Contiene l'applicazione client e la relativa interfaccia grafica JavaFX. Si occupa della presentazione delle informazioni all'utente e dell'invio delle richieste al server.
- ***theknife-server***
 - Contiene l'applicazione server, la gestione delle richieste ricevute dal client, la logica applicativa e di accesso alle risorse esterne, tra cui la base di dati e i servizi di geolocalizzazione.

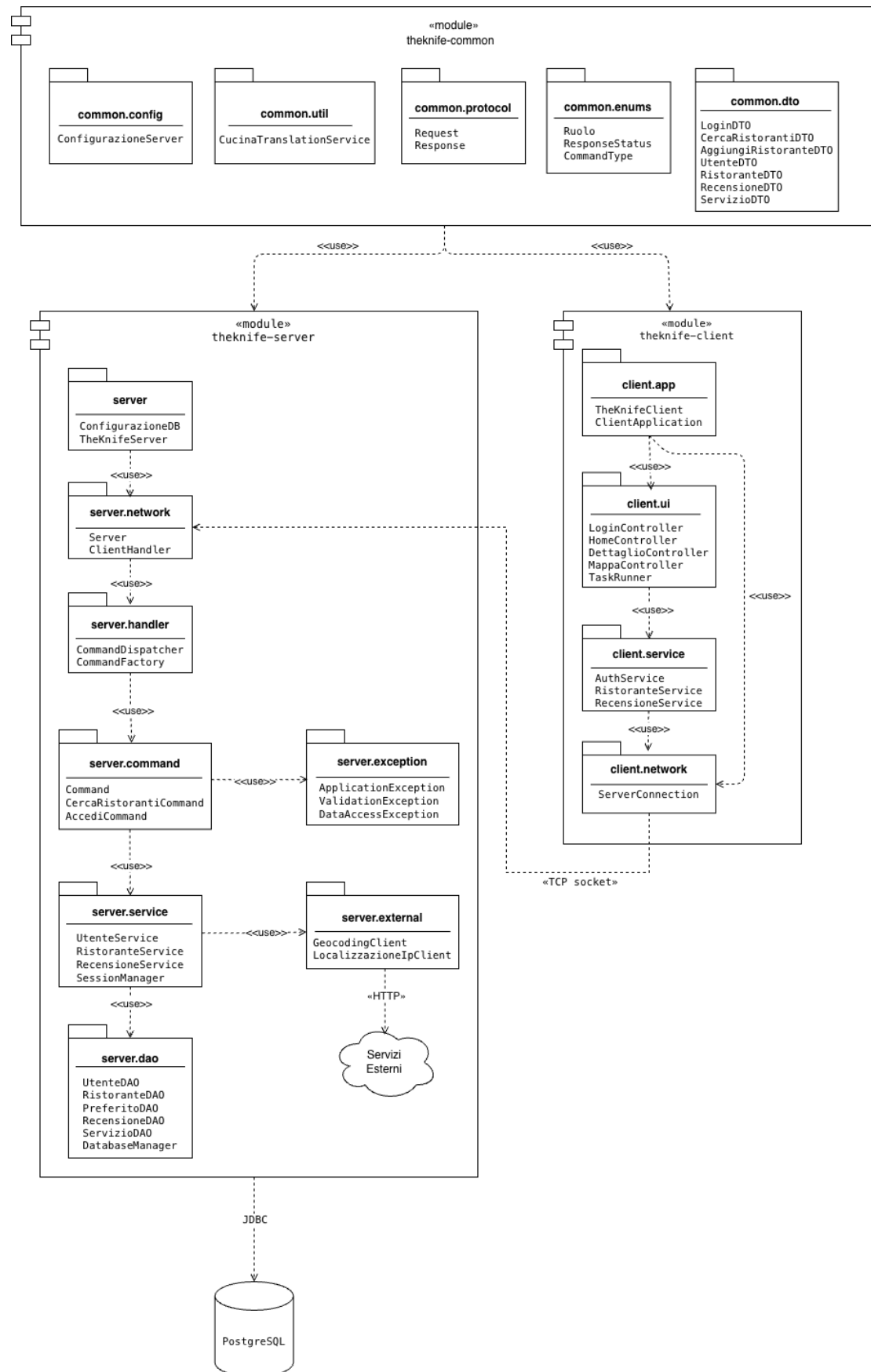


Figura 1 - Package diagram



3.2 Struttura dei moduli

Modulo common

Il modulo *theknife-common* è suddiviso nei seguenti packages:

- *config*
 - Package che contiene la classe *ConfigurazioneServer*, per unificare a livello di sistema i valori predefiniti della porta e dell'host del server.
- *dto*
 - Package che contiene 17 classi corrispondenti ai *Data Transfer Objects*, ovvero gli oggetti che attraversano la rete nelle richieste e nelle risposte in base al protocollo di comunicazione definito.
- *enums*
 - Package che contiene 3 enumerazioni per definire i valori delle costanti di tipo *CommandType* (che definisce il tipo di comando), *ResponseStatus* (che definisce lo stato della risposta) e *Ruolo* (che definisce il ruolo dell'utente autenticato).
- *protocol*
 - Package che contiene le due classi che definiscono il protocollo di comunicazione tra client e server, ovvero *Request* (che rappresenta le richieste inviate dal client al server) e *Response* (che rappresenta le risposte inviate dal server al client).
- *util*
 - Package che contiene la classe di utilità *CucinaTranslationService*, che gestisce la traduzione dei termini di cucina e stile presenti nel dataset.

Modulo client

Il modulo *theknife-client* è suddiviso nei seguenti packages:

- *app*
 - Package che definisce il punto d'ingresso del client: contiene *TheKnifeClient* e *ClientApplication*, che definiscono rispettivamente l'unico punto di ingresso del jar shaded e la logica JavaFX.
- *network*
 - Package che contiene la classe *ServerConnection* che definisce la connessione socket verso il server e fornisce i metodi

inviaRichiestaAsync(Request) (comunicazione asincrona con il server) e *inviaRichiesta(Request)* (comunicazione sincrona con il server).

- *service*
 - Package che contiene i service del client, nello specifico le tre classi *AuthService*, *RecensioneService* e *RistoranteService*, responsabili della conversione delle azioni richieste dalla GUI in richieste da inviare al server.
- *ui*
 - Package che contiene i controller delle schermate JavaFX, uno per ogni file fxml nelle risorse, oltre alle classi di supporto *TaskRunner* (che esegue le chiamate al server fuori dal JavaFX Application Thread), *Toast*, *Responsive* e *CucinaFormatter*.

Modulo server

Il modulo *theknife-server* è suddiviso nei seguenti packages:

- *command*
 - Package che implementa il pattern *Command*, contenente 22 classi (una per ogni *CommandType*). Ogni *Command* verifica che il payload sia del tipo atteso, controlla la proprietà delle risorse toccate quando l'operazione lo richiede, delega la logica di dominio al service e traduce l'esito in una risposta.
- *dao*
 - Package che definisce il layer di accesso ai dati (DAO) del server. Ogni classe incapsula le operazioni sulle tabelle del database *dbTK*: apre una connessione tramite *DatabaseManager*, utilizza *try-with-resources* e riconduce le eccezioni di accesso ai dati a *DataAccessException*. I DAO non contengono logica di dominio: si limitano alle operazioni di lettura e scrittura dei dati, restituendo o utilizzando DTO del modulo *theknife-common*.
- *exception*
 - Package che implementa le eccezioni del server, distinte in base allo stato (*ResponseStatus*) che dovrà essere restituito al client. *ValidationException* se i dati in ingresso non sono validi, *ApplicationException* se i dati sono validi ma l'operazione viola una regola di dominio e *DataAccessException* se il database non risponde o la query fallisce.

- *external*
 - Package contiene i componenti utilizzati dal server per interagire con servizi esterni, in particolare il geocoding diretto e inverso e la localizzazione approssimata tramite indirizzo IP.
- *handler*
 - Package che implementa l'instradamento delle richieste e il controllo degli accessi. *CommandDispatcher* identifica l'utente associato al token di sessione, verifica che il ruolo sia autorizzato a eseguire il comando e delega l'esecuzione al relativo *Command*.
- *network*
 - Package che implementa il livello di rete del server, gestendo il ciclo di vita del socket di ascolto e la comunicazione con i singoli client, mediante la classe *Server* (che verifica il database, costruisce la catena DAO - service - *Command* e accetta le connessioni) e la classe *ClientHandler*, che serve un client per volta su un thread dedicato del pool.
- *service*
 - Package che implementa la logica di dominio del server: validazione dei dati in ingresso, regole di business e orchestrazione dei DAO. Comprende tre service, ognuno dei quali corrisponde ad un'area del protocollo (*RistoranteService*, *RecensioneService*, *UtenteService*), la classe *SessionManager*, il registro delle sessioni attive condiviso tra tutti i thread client e la classe di supporto *EsitoAggiuntaRistorante* (che incapsula l'esito dell'inserimento di un ristorante).

Dipendenze tra i moduli

Il modulo *theknife-common* costituisce il livello condiviso dell'applicazione ed è una dipendenza sia di *theknife-client* sia di *theknife-server*. Esso contiene le classi necessarie a definire il contratto di comunicazione tra le due componenti, inclusi *Request*, *Response*, i DTO, le enumerazioni del protocollo e le configurazioni condivise.

Il modulo *theknife-client* dipende da *theknife-common* per costruire le richieste e interpretare le risposte ricevute dal server, mentre *theknife-server* dipende da *theknife-common* per ricevere le richieste, produrre le risposte e condividere le strutture dati del protocollo.

Non esiste una dipendenza diretta tra *theknife-client* e *theknife-server*: la comunicazione tra i due moduli avviene esclusivamente attraverso il protocollo definito in *theknife-common*.



Le dipendenze sono definite tramite Maven e il modulo *theknife-common* viene incluso come dipendenza da entrambi i moduli applicativi.

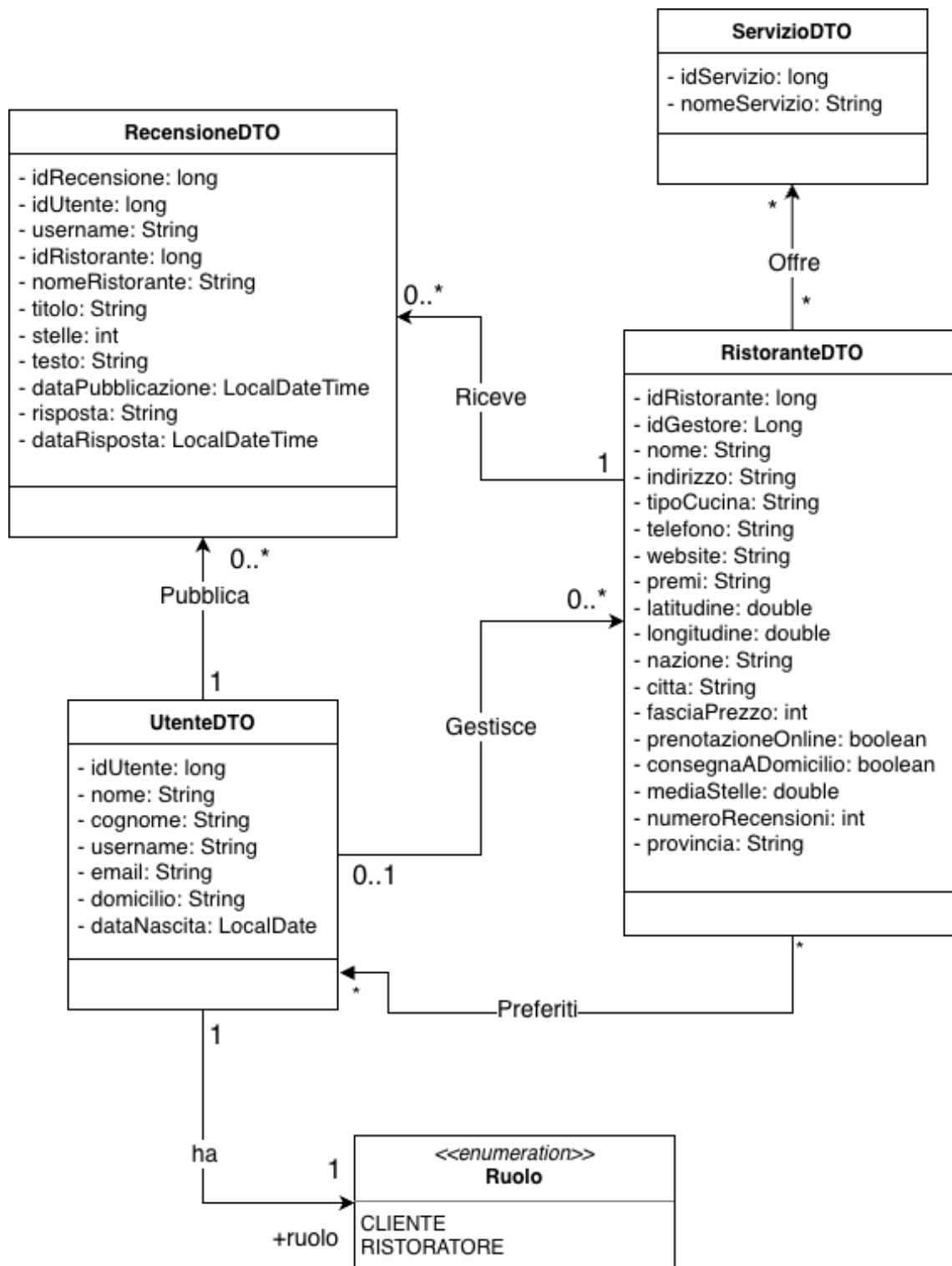


Figura 2 - Class diagram



4. Progettazione del software

4.1 Comunicazione client-server

La comunicazione tra client e server nel progetto è realizzata tramite una connessione socket TCP.

Il server rimane in ascolto su una porta predefinita e accetta le connessioni provenienti dai client. Per ogni connessione viene gestita la comunicazione con il client mediante un'apposita istanza di *ClientHandler*.

Il client stabilisce la connessione all'avvio dell'applicazione e utilizza la connessione per inviare le richieste e ricevere le relative risposte.

Il protocollo applicativo utilizzato per lo scambio dei dati è definito nel modulo *theknife-common* e include le classi *Request* e *Response* che rappresentano rispettivamente le richieste inviate dal client e le risposte restituite dal server.



4.2 Protocollo Request/Response

Il protocollo di comunicazione tra client e server è basato su un modello di richiesta-risposta.

Il client invia una *Request* contenente il comando da eseguire e gli eventuali dati necessari; il server elabora la richiesta e restituisce una *Response* contenente l'esito dell'operazione e gli eventuali dati da restituire al client.

Ogni *Request* contiene:

- *CommandType*, che identifica l'operazione richiesta.
- *payload*, che contiene gli eventuali dati necessari all'esecuzione del comando.
- *sessionToken*, utilizzato per identificare la sessione dell'utente per le operazioni che richiedono autenticazione.
- *indirizzoClient*, contenente l'indirizzo IP del client.

Ogni *Response* contiene:

- *ResponseStatus*, che identifica l'esito dell'operazione.
- *payload*, che contiene gli eventuali dati restituiti dal server.
- *messaggio*, utilizzato per fornire informazioni aggiuntive, in particolare in caso di errore.

Flusso tipico (non tutti i comandi arrivano necessariamente al DAO):

```
Client -> Request -> CommandDispatcher -> CommandFactory -> Command -> Service -> DAO  
-> Service -> Command -> Response -> Client
```

4.3 Gestione dei comandi

Il server utilizza il pattern *Command* per associare ogni richiesta ricevuta a una specifica operazione applicativa. La gestione delle richieste è coordinata dalla classe *CommandDispatcher*, che costituisce il punto di ingresso della logica di elaborazione dei comandi.

CommandDispatcher elabora la richiesta ricevuta, risolve il token di sessione in un utente autenticato quando presente e verifica che il ruolo dell'utente disponga dei permessi necessari per eseguire l'operazione richiesta. Nel caso in cui il controllo di autorizzazione ha esito positivo, *CommandDispatcher* recupera il relativo *Command* tramite *CommandFactory* e ne avvia l'esecuzione.

CommandFactory costruisce i *Command* disponibili nel sistema; le istanze sono costruite una volta sola durante l'avvio del server e successivamente conservate in una mappa immutabile, evitando di ripetere la costruzione ad ogni richiesta.

Il controllo degli accessi viene quindi effettuato prima dell'esecuzione del comando: una richiesta proveniente da un utente privo dei permessi necessari viene rifiutata da *CommandDispatcher* e non raggiunge la logica applicativa del *Command* corrispondente.

Ogni *Command* si occupa delle specifiche verifiche relative all'operazione richiesta, valida il tipo del payload ricevuto e delega l'elaborazione della logica applicativa al *Service* corrispondente. Quando l'operazione richiede l'accesso alla base di dati, il *Service* utilizza il relativo DAO per effettuare le operazioni di lettura o modifica dei dati.

Il risultato dell'elaborazione, infine, è trasformato dal *Command* in una *Response* da restituire al client.

Le eccezioni generate durante l'elaborazione sono gestite dal livello responsabile della costruzione della risposta, traducendo l'esito dell'operazione in un opportuno *ResponseStatus* e in un eventuale messaggio destinato al client.



4.4 Gestione delle sessioni

La gestione delle sessioni è implementata nel modulo theknife-server e consente di associare le richieste provenienti dai client agli utenti precedentemente autenticati.

A seguito di un'autenticazione corretta, il server genera un token di sessione tramite `UUID.randomUUID().toString()`. Il token viene associato all'utente autenticato e restituito al client all'interno di `LoginResultDTO`.

Le sessioni attive sono gestite dalla classe `SessionManager`, che mantiene una mappa concorrente in memoria avente come chiave il token di sessione e come valore l'oggetto `UtenteDTO` associato:

`Token -> UtenteDTO`

L'utilizzo di una `ConcurrentHashMap` consente al registro delle sessioni di essere utilizzato in sicurezza dai diversi thread che gestiscono contemporaneamente le connessioni dei client.

Per le richieste successive all'autenticazione, il client inserisce il token di sessione all'interno dell'oggetto `Request`. Il `CommandDispatcher` recupera il token dalla richiesta e lo utilizza tramite `SessionManager` per risolvere l'utente associato.

Una volta identificato l'utente, il `CommandDispatcher` verifica che il suo ruolo sia compatibile con il livello di accesso richiesto dal comando. Il controllo viene effettuato prima della creazione del relativo `Command`, impedendo quindi che una richiesta non autorizzata raggiunga la logica applicativa.

La gestione della sessione comprende tre operazioni principali:

- **Login** (accesso): il server verifica le credenziali dell'utente, genera un nuovo token e registra la sessione tramite `SessionManager`.
- **Utilizzo della sessione**: il token viene trasmesso nelle richieste successive e utilizzato dal server per identificare l'utente e verificarne le autorizzazioni.
- **Logout** (disconnessione): il token viene rimosso dal `SessionManager`, invalidando la relativa sessione.

Le sessioni sono mantenute **esclusivamente in memoria** e non vengono persistite nella base di dati. Di conseguenza, il riavvio del server comporta la perdita di tutte le sessioni attive e richiede agli utenti di autenticarsi nuovamente.

Sul lato client, `ServerConnection` mantiene il token della sessione corrente e l'oggetto `UtenteDTO` corrispondente. Al logout, questi dati vengono azzerati, mentre la connessione socket con il server rimane attiva.



Un flusso di autenticazione e utilizzo della sessione:

1. Il client invia una richiesta di autenticazione contenente le credenziali dell'utente.
2. *UtenteService* verifica le credenziali confrontando la password fornita con l'hash BCrypt memorizzato nella base di dati.
3. In caso di autenticazione corretta, viene generato un token casuale tramite UUID.
4. *SessionManager* associa il token all'utente autenticato.
5. Il token è restituito al client.
6. Il client inserisce il token nelle richieste successive.
7. *CommandDispatcher* risolve il token tramite *SessionManager* e recupera l'utente associato.
8. *CommandDispatcher* verifica il requisito di accesso del comando in base al ruolo dell'utente.
9. Se l'accesso è consentito, la richiesta è inoltrata al *Command*.
10. Al logout (disconnessione), *SessionManager* rimuove il token e la relativa sessione.

Schematicamente:

```
Client (Request + sessionToken) -> CommandDispatcher (getUtenteFromSession(token) -> SessionManager (UtenteDTO) -> CommandDispatcher (verifica ruolo) -> CommandFactory -> Command -> Service -> DAO -> Base di dati -> Service -> Command -> Response -> Client
```

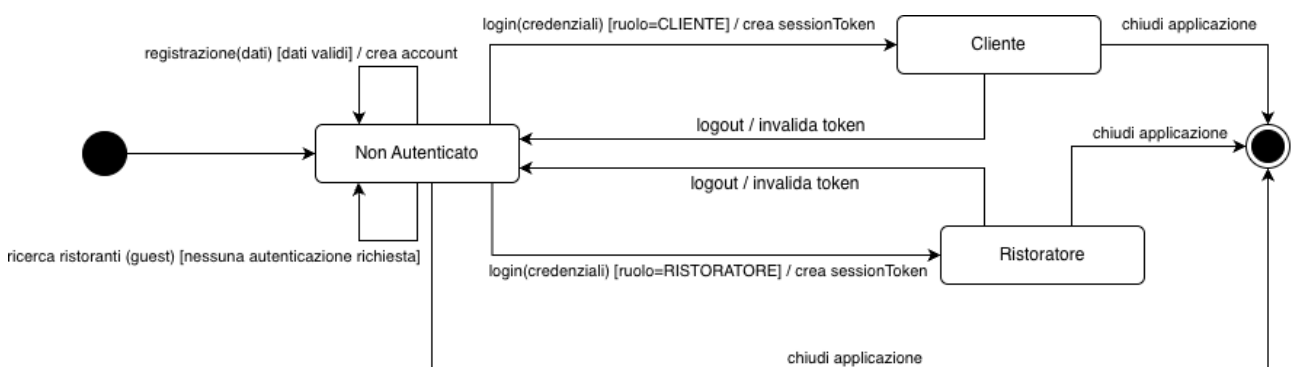


Figura 3 - State diagram

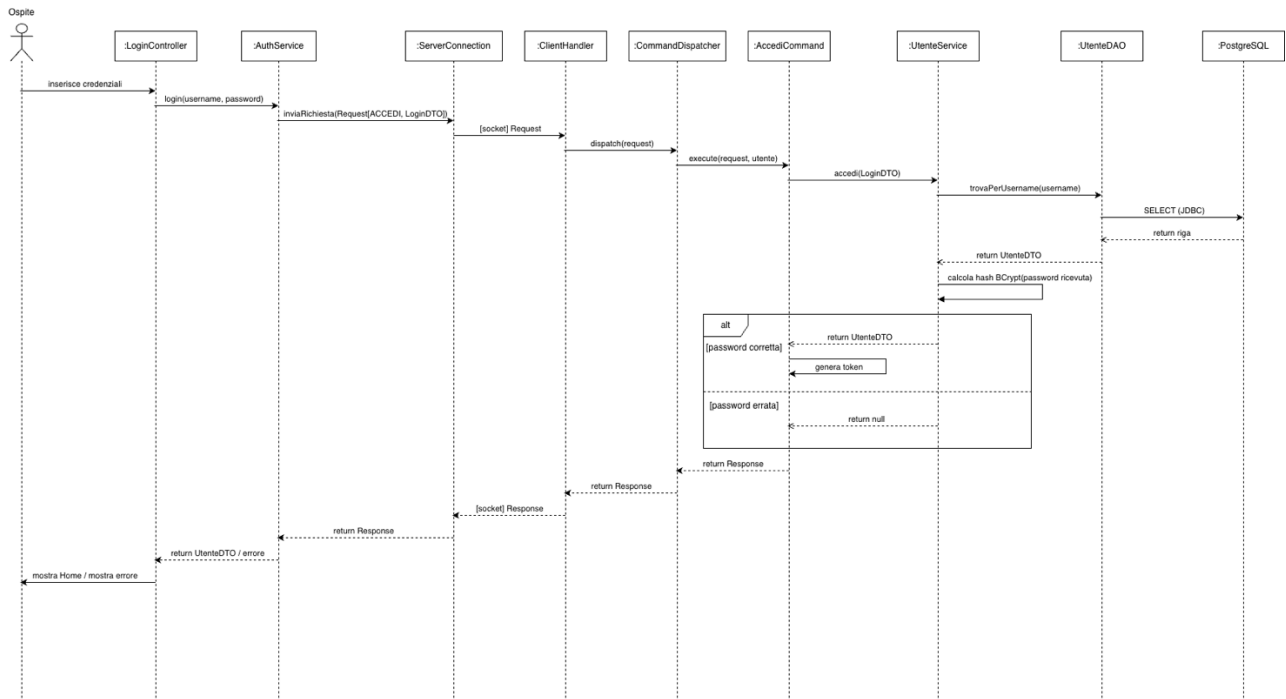


Figura 4 - Sequence diagram

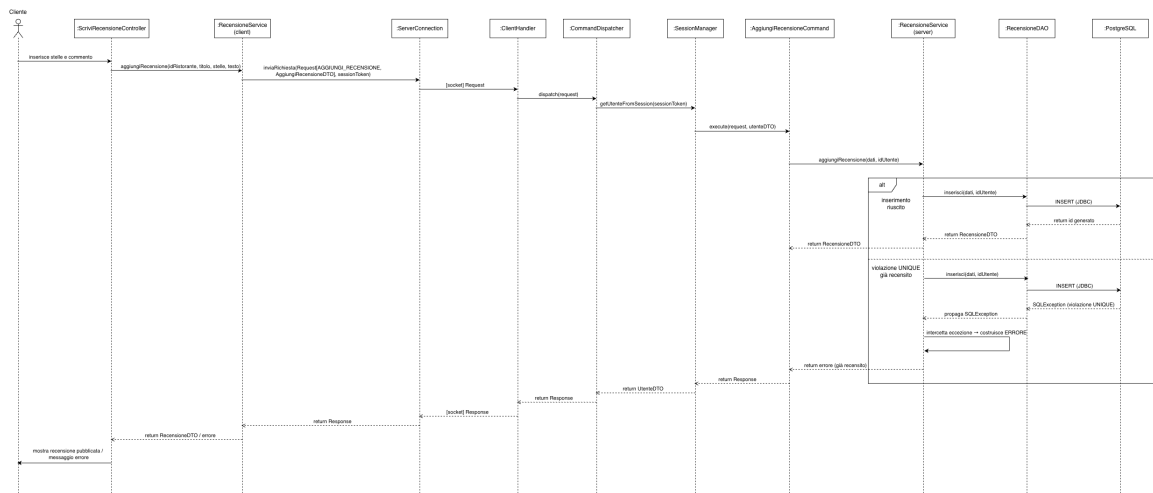


Figura 5 - Sequence diagram (Aggiungi Recensione)





5. Progettazione della base di dati

5.1 Modello concettuale

Il modello concettuale della base di dati è stato progettato per rappresentare le principali entità coinvolte nell'applicazione e le relazioni tra di esse, con particolare riferimento alla gestione degli utenti, dei ristoranti, dei servizi, delle recensioni e dei ristoranti preferiti. Il modello è stato successivamente tradotto in uno schema relazionale, implementato mediante il sistema di gestione di basi di dati PostgreSQL, costituito dalle seguenti tabelle:

- Tabella *Utenti*: contiene le informazioni relative agli utenti dell'applicazione. Ogni utente è identificato da un identificativo univoco (*id*) e include i seguenti campi:
 - *username*: il nome utente, univoco e obbligatorio.
 - *password*: la password dell'utente, memorizzata sotto forma di hash BCrypt.
 - *nome*.
 - *cognome*.
 - *email*: l'indirizzo email dell'utente, che, se specificato, deve essere univoco.
 - *data_nascita*: la data di nascita dell'utente.
 - *domicilio*: il domicilio dell'utente.
 - *ruolo*: il ruolo dell'utente, che può essere CLIENTE o RISTORATORE.
- Tabella *Servizio*: contiene le informazioni relative ai servizi offerti dai ristoranti. Ogni servizio è identificato da un identificativo univoco (*id*) e include il campo *nome*, che identifica il nome del servizio, che deve essere univoco.
- Tabella *RistorantiTheKnife*: contiene le informazioni relative ai ristoranti gestiti dall'applicazione. Ogni ristorante è identificato da un identificativo univoco (*id_ristorante*) e include i seguenti campi:
 - *nome*: il nome del ristorante.
 - *nazione*: la nazione in cui si trova il ristorante.
 - *città*: la città in cui si trova il ristorante.



- *provincia*: la provincia in cui si trova il ristorante.
- *indirizzo*: l'indirizzo del ristorante.
- *latitudine*: la latitudine della posizione del ristorante.
- *longitudine*: la longitudine della posizione del ristorante.
- *fascia_prezzo*: la fascia di prezzo del ristorante, con valori compresi tra 1 e 4.
- *prenotazione_online*: indica se il ristorante offre la prenotazione online.
- *consegna_a_domicilio*: indica se il ristorante offre la consegna a domicilio.
- *tipo_cucina*: il tipo di cucina del ristorante.
- *telefono*: il numero di telefono del ristorante.
- *website*: il sito web del ristorante.
- *premi*: i premi o riconoscimenti del ristorante.
- *id_gestore*: l'identificativo dell'utente che gestisce il ristorante, utilizzato come chiave esterna verso *Utenti*.
Il valore può essere nullo e, in caso di eliminazione dell'utente, viene impostato a NULL.

- Tabella *RistoranteServizio*: rappresenta l'associazione molti-a-molti tra i ristoranti e i servizi offerti. La chiave primaria composta da *id_ristorante* e *id_servizio* impedisce duplicazioni della stessa associazione.

Ogni tupla della tabella include i seguenti campi:

- *id_ristorante*: l'identificativo del ristorante, che fa riferimento alla tabella *RistorantiTheKnife*.
- *id_servizio*: l'identificativo del servizio, che fa riferimento alla tabella *Servizio*.

- Tabella *Recensioni*: contiene le informazioni relative alle recensioni dei ristoranti.

Ogni recensione è identificata da un identificativo univoco (*id_recensione*) e include i seguenti campi:

- *id_ristorante*: l'identificativo del ristorante, che fa riferimento alla tabella *RistorantiTheKnife*.
- *id_cliente*: l'identificativo del cliente, che fa riferimento alla tabella *Utenti*.
- *titolo*: il titolo della recensione.
- *testo*: il testo della recensione.

- *stelle*: il numero di stelle assegnate al ristorante, con valori compresi tra 1 e 5.
- *data_pubblicazione*: la data di pubblicazione della recensione.
- *risposta*: la risposta del gestore del ristorante alla recensione.
- *data_risposta*: la data di pubblicazione della risposta del gestore del ristorante.

La tabella *Recensioni* impone inoltre che uno stesso cliente non possa inserire più di una recensione per lo stesso ristorante.

La risposta del gestore e la relativa data possono invece essere assenti, in quanto una recensione può inizialmente non aver ricevuto una risposta.

- Tabella *Preferiti*: rappresenta l'associazione molti-a-molti tra i clienti e i ristoranti preferiti. La chiave primaria composta da *id_cliente* e *id_ristorante* impedisce duplicazioni della stessa associazione. Ogni tupla della tabella include i seguenti campi:
 - *id_cliente*: l'identificativo del cliente, che fa riferimento alla tabella *Utenti*.
 - *id_ristorante*: l'identificativo del ristorante, che fa riferimento alla tabella *RistorantiTheKnife*.

Le **associazioni** presenti tra le entità sono:

- **Utente → Ristorante**: un utente con ruolo RISTORATORE può gestire più ristoranti; ogni ristorante può avere un gestore, identificato dal campo *id_gestore*.
- **Ristorante ↔ Servizio**: associazione molti-a-molti, realizzata tramite la relazione *RistoranteServizio*.
- **Cliente → Recensione**: un cliente può scrivere più recensioni.
 - Nello specifico, il vincolo *UNIQUE (id_cliente, id_ristorante)* imposto dallo schema stabilisce che ogni cliente può inserire al massimo una recensione per ciascun ristorante.
- **Ristorante → Recensione**: un ristorante può ricevere più recensioni.
- **Cliente ↔ Ristorante**: associazione molti-a-molti per i preferiti, realizzata tramite la relazione *Preferiti*.



L'**integrità referenziale** tra le entità è garantita mediante chiavi esterne. Sono stati inoltre definiti specifici comportamenti in caso di cancellazione: le recensioni e i preferiti associati a un ristorante o a un cliente sono eliminati automaticamente, così come le associazioni tra ristoranti e servizi, quando viene eliminato uno dei relativi elementi. Nel caso dell'eliminazione di un gestore, invece, il riferimento al gestore viene impostato a NULL, mantenendo il ristorante nella base di dati.



5.2 Diagramma ER

Il diagramma ER (Entity-Relationship) della base di dati rappresenta le principali entità coinvolte nell'applicazione, i relativi attributi e le associazioni che intercorrono tra di esse. Esso costituisce la rappresentazione del modello concettuale della base di dati e descrive, a un livello indipendente dai dettagli implementativi dello schema relazionale, le informazioni gestite dall'applicazione e i legami esistenti tra tali informazioni.

Il diagramma ER include le seguenti entità:

- *Utente*: rappresenta gli utenti dell'applicazione, che possono assumere il ruolo di cliente o di ristoratore.
- *Servizio*: rappresenta i servizi che possono essere offerti dai ristoranti.
- *Ristorante*: rappresenta i ristoranti presenti nell'applicazione.
- *Recensione*: rappresenta le recensioni pubblicate dai clienti relativamente ai ristoranti.

Il concetto di ristorante preferito non costituisce un'entità autonoma, ma è rappresentato dall'associazione tra l'entità *Utente* e l'entità *Ristorante*, che permette a un cliente di associare uno o più ristoranti ai propri preferiti.

L'entità *Utente* rappresenta gli utenti dell'applicazione. I suoi attributi sono:

- *id*: identificativo univoco dell'utente;
- *username*: nome utente;
- *password*: password dell'utente, memorizzata sotto forma di hash BCrypt;
- *nome*: nome dell'utente;
- *cognome*: cognome dell'utente;
- *email*: indirizzo email dell'utente;
- *data_nascita*: data di nascita dell'utente;
- *domicilio*: domicilio dell'utente;
- *ruolo*: ruolo dell'utente, che può essere CLIENTE o RISTORATORE.

L'entità *Servizio* rappresenta i servizi offerti dai ristoranti. I suoi attributi sono:

- *id*: identificativo univoco del servizio;
- *nome*: nome del servizio.

L'entità *Ristorante* rappresenta i ristoranti gestiti dall'applicazione. I suoi attributi sono:

- *id_ristorante*: identificativo univoco del ristorante;
- *nome*: nome del ristorante;
- *nazione*: nazione in cui si trova il ristorante;

- *citta*: città in cui si trova il ristorante;
- *provincia*: provincia in cui si trova il ristorante;
- *indirizzo*: indirizzo del ristorante;
- *latitudine*: latitudine della posizione del ristorante;
- *longitudine*: longitudine della posizione del ristorante;
- *fascia_prezzo*: fascia di prezzo del ristorante, con valori compresi tra 1 e 4;
- *prenotazione_online*: indica se il ristorante offre la prenotazione online;
- *consegna_a_domicilio*: indica se il ristorante offre la consegna a domicilio;
- *tipo_cucina*: tipologia di cucina del ristorante;
- *telefono*: numero di telefono del ristorante;
- *website*: sito web del ristorante;
- *premi*: premi o riconoscimenti del ristorante.

L'entità *Recensione* rappresenta le recensioni pubblicate dai clienti sui ristoranti. I suoi attributi sono:

- *id_recensione*: identificativo univoco della recensione;
- *titolo*: titolo della recensione;
- *testo*: testo della recensione;
- *stelle*: valutazione assegnata al ristorante, con valori compresi tra 1 e 5;
- *data_pubblicazione*: data di pubblicazione della recensione;
- *risposta*: eventuale risposta del gestore del ristorante;
- *data_risposta*: eventuale data di pubblicazione della risposta.

Le entità individuate sono collegate dalle seguenti associazioni:

- **Gestisce**, tra *Utente* e *Ristorante*: un utente con ruolo RISTORATORE può gestire zero, uno o più ristoranti; ogni ristorante può avere zero oppure un gestore, identificato nel modello relazionale dal campo *id_gestore*. L'eventuale assenza del gestore è consentita dallo schema relazionale, che permette un valore NULL per il riferimento al gestore.
- **Offre**, tra *Ristorante* e *Servizio*: un ristorante può offrire zero, uno o più servizi e uno stesso servizio può essere offerto da zero, uno o più ristoranti. Si tratta pertanto di un'associazione molti-a-molti, implementata nello schema relazionale mediante la tabella associativa *RistoranteServizio*.
- **Pubblica**, associazione ternaria tra *Utente*, *Ristorante* e *Recensione*: rappresenta la pubblicazione di una recensione da parte di un utente relativamente ad un determinato ristorante. L'associazione permette quindi di collegare ciascuna recensione sia all'utente che l'ha pubblicata sia al ristorante

a cui essa si riferisce.

Il vincolo *UNIQUE* (*id_cliente*, *id_ristorante*) presente nello schema relazionale stabilisce inoltre che uno stesso cliente può scrivere al massimo una recensione per ciascun ristorante.

- **Riceve**, tra *Ristorante* e *Recensione*: un ristorante può ricevere zero, una o più recensioni, mentre ogni recensione è riferita a un solo ristorante.
- **Preferisce**, tra *Utente* e *Ristorante*: un cliente può associare zero, uno o più ristoranti ai propri preferiti e un ristorante può essere presente tra i preferiti di zero, uno o più clienti. Si tratta quindi di un'associazione molti-a-molti, implementata nello schema relazionale mediante la tabella associativa *Preferiti*.

L'associazione *Pubblica* permette quindi di rappresentare il collegamento tra una recensione, il cliente che l'ha pubblicata e il ristorante a cui essa si riferisce. La risposta alla recensione non è modellata come una nuova entità, ma come attributo dell'entità *Recensione*, in quanto nello schema della base di dati è previsto un solo campo per la risposta e una sola relativa data.

Il diagramma ER rappresenta quindi una struttura focalizzata sulle entità *Utente*, *Ristorante*, *Servizio* e *Recensione*.

L'entità *Utente* può partecipare a differenti associazioni in funzione del proprio ruolo: come **RISTORATORE** può gestire uno o più ristoranti, mentre come **CLIENTE** può scrivere recensioni e indicare ristoranti come preferiti.

L'entità *Ristorante* è collegata ai servizi che offre, alle recensioni che riceve e agli eventuali utenti che lo hanno inserito tra i propri preferiti.

L'entità *Recensione* collega invece un cliente a un ristorante e contiene, oltre al contenuto della recensione e alla valutazione, l'eventuale risposta del gestore.

Le associazioni molti-a-molti tra *Ristorante* e *Servizio* e tra *Utente* e *Ristorante* per la gestione dei preferiti vengono successivamente tradotte nello schema relazionale mediante le tabelle associative *RistoranteServizio* e *Preferiti*.

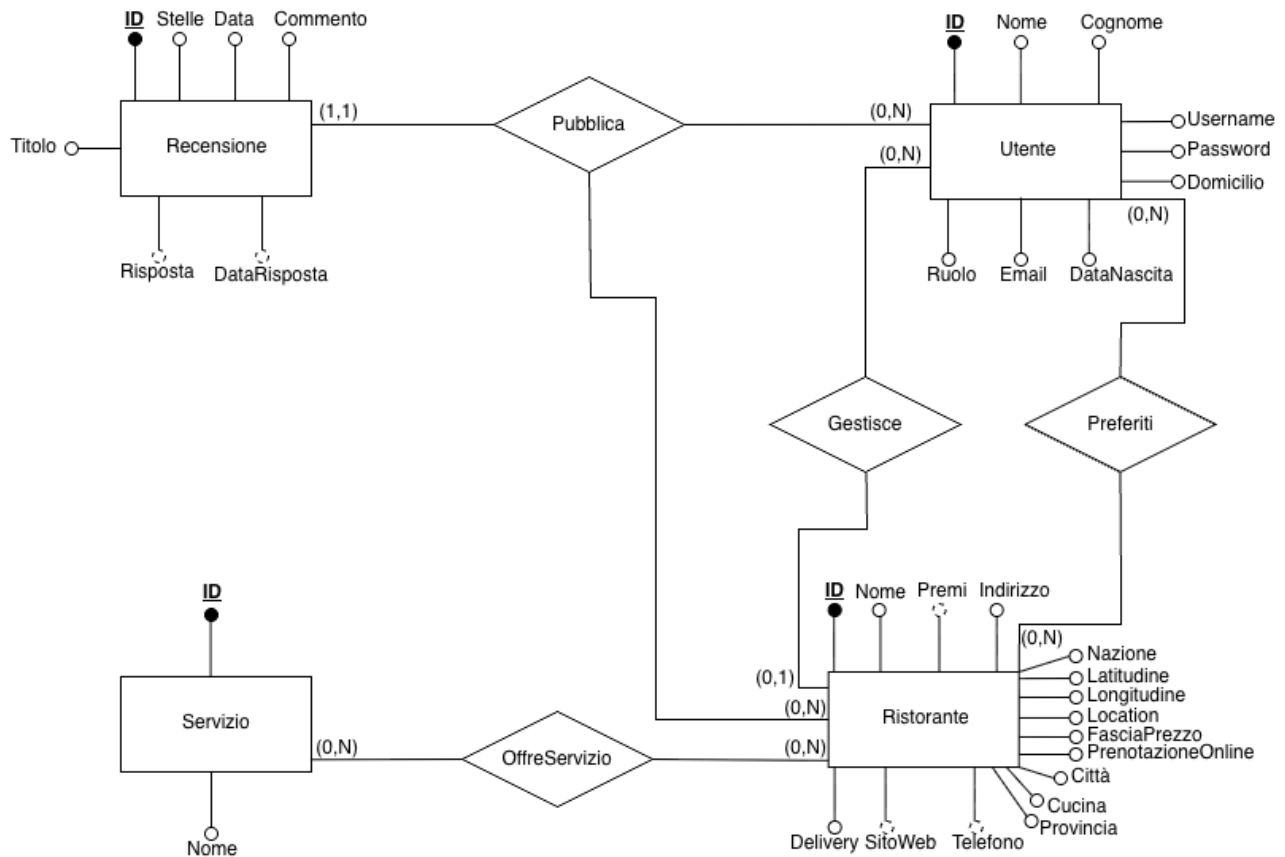


Figura 7 - Diagramma ER

5.3 Schema della base di dati

Lo schema della base di dati rappresenta la traduzione del modello concettuale in una struttura relazionale e definisce le tabelle, le chiavi, i vincoli di integrità e le relazioni tra i dati necessari a supportare le funzionalità principali dell'applicazione. Lo schema è stato implementato utilizzando il sistema di gestione di base di dati PostgreSQL ed è costituito dalle seguenti tabelle:

- Tabella *Utenti*: contiene le informazioni relative agli utenti dell'applicazione. Ogni utente è identificato da un identificativo univoco (*id*) e include i seguenti campi:
 - *id*: l'identificativo univoco dell'utente, generato automaticamente mediante una sequenza PostgreSQL.
 - *username*: il nome utente, obbligatorio e univoco.
 - *password*: la password dell'utente, memorizzata sotto forma di hash BCrypt.
 - *nome*: il nome dell'utente, obbligatorio.
 - *cognome*: il cognome dell'utente, obbligatorio.
 - *email*: l'indirizzo email dell'utente, che, se specificato, deve essere univoco.
 - *data_nascita*: la data di nascita dell'utente.
 - *domicilio*: il domicilio dell'utente.
 - *ruolo*: il ruolo dell'utente, che può assumere i valori CLIENTE o RISTORATORE.
- Tabella *Servizio*: contiene le informazioni relative ai servizi offerti dai ristoranti. Ogni servizio è identificato da un identificativo univoco (*id*) e include i seguenti campi:
 - *id*: l'identificativo univoco del servizio, generato automaticamente mediante una sequenza PostgreSQL.
 - *nome*: il nome del servizio, obbligatorio e univoco.
- Tabella *RistorantiTheKnife*: contiene le informazioni relative ai ristoranti gestiti dall'applicazione. Ogni ristorante è identificato da un identificativo univoco (*id_ristorante*) e include i seguenti campi:
 - *id_ristorante*: l'identificativo univoco del ristorante, generato automaticamente mediante una sequenza PostgreSQL.
 - *nome*: il nome del ristorante, obbligatorio.
 - *nazione*: la nazione in cui si trova il ristorante.
 - *città*: la città in cui si trova il ristorante.
 - *provincia*: la provincia in cui si trova il ristorante.



- *indirizzo*: l'indirizzo del ristorante.
- *latitudine*: la latitudine della posizione del ristorante.
- *longitudine*: la longitudine della posizione del ristorante.
- *fascia_prezzo*: la fascia di prezzo del ristorante, con valori compresi tra 1 e 4.
- *prenotazione_online*: indica se il ristorante offre la prenotazione online. Se non specificato, il valore predefinito è FALSE.
- *consegna_a_domicilio*: indica se il ristorante offre la consegna a domicilio. Se non specificato, il valore predefinito è FALSE.
- *tipo_cucina*: tipologia di cucina del ristorante.
- *telefono*: il numero di telefono del ristorante.
- *website*: il sito web del ristorante.
- *premi*: i premi o riconoscimenti del ristorante.
- *id_gestore*: l'identificativo dell'utente che gestisce il ristorante, utilizzato come chiave esterna (FK) verso la tabella *Utenti*. Il valore può essere nullo e, in caso di eliminazione dell'utente referenziato, è impostato automaticamente a NULL (*ON DELETE SET NULL*).
- Tabella *RistoranteServizio*: rappresenta la tabella associativa utilizzata per implementare nello schema relazionale l'associazione molti-a-molti tra i ristoranti e i servizi offerti. La chiave primaria (PK) composta dai campi *id_ristorante* e *id_servizio* impedisce la duplicazione della stessa associazione. Ogni tupla della tabella include i seguenti campi:
 - *id_ristorante*: l'identificativo del ristorante, chiave esterna (FK) verso la tabella *RistorantiTheKnife*.
La cancellazione di un ristorante comporta automaticamente la cancellazione delle relative associazioni nella tabella *RistoranteServizio* (*ON DELETE CASCADE*).
 - *id_servizio*: l'identificativo del servizio, chiave esterna (FK) verso la tabella *Servizio*.
La cancellazione di un servizio comporta automaticamente la cancellazione delle relative associazioni nella tabella *RistoranteServizio* (*ON DELETE CASCADE*).
- Tabella *Recensioni*: contiene le informazioni relative alle recensioni dei ristoranti. Ogni recensione è identificata da un identificativo univoco (*id_recensione*) e include i seguenti campi:
 - *id_recensione*: l'identificativo univoco della recensione, generato automaticamente mediante una sequenza PostgreSQL.



- *id_ristorante*: l'identificativo del ristorante, chiave esterna (FK) verso la tabella *RistorantiTheKnife*.
La cancellazione di un ristorante comporta automaticamente la cancellazione delle relative recensioni nella tabella *Recensioni* (*ON DELETE CASCADE*).
- *id_cliente*: l'identificativo dell'utente che pubblica la recensione, chiave esterna (FK) verso la tabella *Utenti*.
La cancellazione di un cliente comporta automaticamente la cancellazione delle relative recensioni nella tabella *Recensioni* (*ON DELETE CASCADE*).
- *titolo*: il titolo della recensione.
- *testo*: il testo della recensione.
- *stelle*: il numero di stelle assegnate al ristorante, con valori compresi tra 1 e 5.
- *data_pubblicazione*: la data e l'ora di pubblicazione della recensione, impostata automaticamente al momento dell'inserimento (*CURRENT_TIMESTAMP*) se non specificata.
- *risposta*: l'eventuale risposta del gestore del ristorante alla recensione.
- *data_risposta*: la data e l'ora di pubblicazione della risposta del gestore del ristorante.

La tabella *Recensioni*, inoltre, impone mediante il vincolo *UNIQUE* (*id_cliente*, *id_ristorante*), che uno stesso cliente non possa inserire più di una recensione per lo stesso ristorante. La risposta del gestore e la relativa data possono essere assenti.

- Tabella *Preferiti*: rappresenta la tabella associativa utilizzata per implementare nello schema relazionale l'associazione molti-a-molti tra i clienti e i ristoranti preferiti. La chiave primaria (PK) composta dai campi *id_cliente* e *id_ristorante* impedisce la duplicazione della stessa associazione.

Ogni tupla della tabella include i seguenti campi:

- *id_cliente*: l'identificativo del cliente, chiave esterna verso la tabella *Utenti*.
La cancellazione di un cliente comporta automaticamente la cancellazione delle relative associazioni nella tabella *Preferiti* (*ON DELETE CASCADE*).
- *id_ristorante*: l'identificativo del ristorante, chiave esterna verso la tabella *RistorantiTheKnife*.
La cancellazione di un ristorante comporta automaticamente la



cancellazione delle relative associazioni nella tabella *Preferiti* (*ON DELETE CASCADE*).

Lo schema relazionale utilizza chiavi primarie per garantire l'identificazione univoca delle tuple e chiavi esterne per garantire l'integrità referenziale tra le tabelle.

Sono inoltre presenti vincoli di unicità e di controllo sui valori ammessi. In particolare, il ruolo di un utente è limitato ai valori *CLIENTE* e *RISTORATORE*, la fascia di prezzo di un ristorante è compresa tra 1 e 4 e la valutazione di una recensione è compresa tra 1 e 5.

Sono stati inoltre definiti specifici comportamenti in caso di cancellazione. Le recensioni e i preferiti associati a un ristorante o a un cliente vengono eliminati automaticamente, così come le associazioni tra ristoranti e servizi, quando viene eliminato uno degli elementi coinvolti. Nel caso dell'eliminazione dell'utente associato come gestore di un ristorante, invece, il riferimento viene impostato a *NULL*, mantenendo il ristorante nella base di dati.

Permessi

Per consentire all'applicazione di operare sulla base di dati è stato definito il ruolo applicativo *tk_app*. Al ruolo sono stati assegnati i seguenti privilegi:

- ***SELECT, INSERT, UPDATE, DELETE*** sulle **tabelle**: consentono rispettivamente di leggere, inserire, modificare ed eliminare i dati presenti nelle tabelle dello schema *public*;
- ***USAGE, SELECT*** sulle **sequenze**: consentono di utilizzare le sequenze associate ai campi identificativi generati automaticamente e di leggerne i valori;
- ***USAGE*** sullo **schema *public***: consente al ruolo di accedere agli oggetti presenti nello schema.

I privilegi sulle tabelle e sulle sequenze vengono assegnati mediante ***GRANT ... ON ALL TABLES IN SCHEMA public*** e ***GRANT ... ON ALL SEQUENCES IN SCHEMA public***, mentre il privilegio sullo schema viene assegnato mediante ***GRANT USAGE ON SCHEMA public***.

5.4 Vincoli e integrità dei dati

La progettazione della base di dati include una serie di vincoli e meccanismi di integrità dei dati per garantire la coerenza e la correttezza delle informazioni memorizzate. Questi vincoli sono stati implementati utilizzando le funzionalità di PostgreSQL.

Vincoli di integrità referenziale

I vincoli di integrità referenziale sono utilizzati per garantire che le relazioni tra le tabelle siano valide. Le chiavi esterne sono utilizzate per collegare le tabelle e le azioni di cascata sono utilizzate per gestire le eliminazioni delle tuple.

Questi vincoli sono implementati utilizzando:

- **Chiavi esterne:**
 - *id_gestore* nella tabella *RistorantiTheKnife* fa riferimento alla chiave primaria *id* nella tabella *Utenti*.
 - *id_ristorante* nella tabella *RistoranteServizio* fa riferimento alla chiave primaria *id_ristorante* nella tabella *RistorantiTheKnife*.
 - *id_servizio* nella tabella *RistoranteServizio* fa riferimento alla chiave primaria *id* nella tabella *Servizio*.
 - *id_ristorante* nella tabella *Recensioni* fa riferimento alla chiave primaria *id_ristorante* nella tabella *RistorantiTheKnife*.
 - *id_cliente* nella tabella *Recensioni* fa riferimento alla chiave primaria *id* nella tabella *Utenti*.
 - *id_cliente* nella tabella *Preferiti* fa riferimento alla chiave primaria *id* nella tabella *Utenti*.
 - *id_ristorante* nella tabella *Preferiti* fa riferimento alla chiave primaria *id_ristorante* nella tabella *RistorantiTheKnife*.
- **Azioni di cascata:**
 - **ON DELETE CASCADE:** la cancellazione di un ristorante comporta automaticamente la cancellazione delle relative tuple nelle tabelle *RistoranteServizio*, *Recensioni* e *Preferiti*. Analogamente, la cancellazione di un servizio elimina le relative associazioni in *RistoranteServizio*, mentre la cancellazione di un cliente elimina le relative recensioni e preferenze.
 - **ON DELETE SET NULL:** la cancellazione di un utente referenziato come gestore di un ristorante comporta l'impostazione a NULL del relativo campo *id_gestore*, mantenendo il ristorante nella base di dati.

Vincoli di integrità delle chiavi primarie

I vincoli di integrità delle chiavi primarie sono utilizzati per garantire che ogni tupla in una tabella abbia un identificativo univoco. Le chiavi primarie garantiscono quindi l'identificazione delle tuple e, di conseguenza, non possono assumere valori NULL.

Questi vincoli sono implementati utilizzando le chiavi primarie e includono:

- *id* nella tabella *Utenti*.
- *id* nella tabella *Servizio*.
- *id_ristorante* nella tabella *RistorantiTheKnife*.
- *id_recensione* nella tabella *Recensioni*.
- *id_cliente* e *id_ristorante* nella tabella *Preferiti* (PK composta).
- *id_ristorante* e *id_servizio* nella tabella *RistoranteServizio* (PK composta).

Vincoli di unicità

I vincoli di unicità sono utilizzati per garantire che determinati valori, singolarmente o in combinazione, non siano duplicati.

Questi vincoli sono implementati utilizzando le chiavi univoche e includono:

- *username* nella tabella *Utenti*.
- *email* nella tabella *Utenti*, se valorizzata, deve essere univoca.
- *nome* nella tabella *Servizio*.
- Il vincolo di unicità composto *UNIQUE (id_cliente, id_ristorante)* nella tabella *Recensioni* impedisce ad uno stesso cliente di inserire più di una recensione per lo stesso ristorante.

Vincoli di integrità dei valori

I vincoli di integrità dei valori sono utilizzati per garantire che i valori in una colonna siano validi. Questi vincoli sono implementati utilizzando i vincoli di controllo e includono:

- *ruolo* nella tabella *Utenti* può essere solo *CLIENTE* o *RISTORATORE*.
- *fascia_prezzo* nella tabella *RistorantiTheKnife* può essere solo 1, 2, 3 o 4.
- *stelle* nella tabella *Recensioni* può essere solo 1, 2, 3, 4 o 5.

Gestione delle transazioni

La gestione delle transazioni è realizzata a livello applicativo mediante le funzionalità JDBC e si basa sul supporto transazionale fornito da PostgreSQL. I DAO utilizzano connessioni alla base di dati ottenute tramite *DatabaseManager*; nella maggior parte



dei casi, ogni metodo esegue una singola operazione SQL e la connessione viene chiusa al termine dell'operazione mediante *try-with-resources*.

Nei casi in cui un'operazione richiede l'esecuzione di più istruzioni SQL che devono essere considerate come un'unica operazione atomica, viene utilizzata una transazione esplicita.

In particolare, nel metodo *associa()* della classe *ServizioDAO* nel package *theknife.server.dao*, la sostituzione dei servizi associati a un ristorante prevede:

- la disattivazione dell'auto-commit mediante *setAutoCommit(false)*;
- la cancellazione delle associazioni precedenti nella tabella *RistoranteServizio*;
- l'inserimento delle nuove associazioni;
- il COMMIT della transazione al termine delle operazioni;
- il ROLLBACK in caso di errore, in modo da annullare le modifiche effettuate durante la transazione.

In questo modo si garantisce che la sostituzione delle associazioni avvenga in maniera atomica: le modifiche vengono rese permanenti solo se tutte le operazioni hanno esito positivo; in caso contrario, le modifiche effettuate vengono annullate.

Le operazioni che prevedono una singola istruzione SQL, come l'inserimento o la cancellazione di un singolo ristorante, non richiedono invece una transazione esplicita nel codice DAO. Tali operazioni vengono eseguite utilizzando il comportamento transazionale predefinito della connessione JDBC.

Controllo degli accessi

Per limitare le operazioni effettuabili dall'applicazione sulla base di dati è stato definito il ruolo applicativo *tk_app*. A tale ruolo sono stati assegnati l'accesso alle tabelle e alle sequenze necessarie per il funzionamento dell'applicazione.

I permessi includono:

- *SELECT, INSERT, UPDATE, DELETE* sulle tabelle.
- *USAGE, SELECT* sulle sequenze.
- *USAGE* sullo schema public.

6. Scelte progettuali

6.1 Gestione dei dati e persistenza

La gestione dei dati e la persistenza sono state progettate per garantire una separazione delle responsabilità tra accesso ai dati e logica applicativa, nonché per assicurare l'integrità e la corretta gestione delle operazioni sulla base di dati.

La persistenza dei dati è affidata al sistema di gestione di basi di dati PostgreSQL, mentre l'accesso ai dati è organizzato mediante il pattern **Data Access Object (DAO)**. La logica applicativa e di business è invece concentrata nel livello **Service**, mantenendo separate le responsabilità dei diversi livelli dell'applicazione.

Gestione dei dati

La gestione dei dati è implementata mediante il pattern **Data Access Object (DAO)**. Il pattern DAO permette di separare le operazioni di accesso alla base di dati dalla logica applicativa, centralizzando le interrogazioni SQL e le operazioni di persistenza all'interno di classi dedicate. Questa organizzazione favorisce la manutenibilità del codice e riduce l'accoppiamento tra il livello applicativo e il sistema di gestione della base di dati.

Pattern DAO

Il pattern DAO è implementato nel modulo *theknife-server* e comprende DAO dedicati alle principali entità della base di dati, tra cui:

- *UtenteDAO*: gestisce l'accesso ai dati relativi agli utenti.
- *ServizioDAO*: gestisce l'accesso ai dati relativi ai servizi e alle associazioni tra ristoranti e servizi.
- *RistoranteDAO*: gestisce l'accesso ai dati relativi ai ristoranti e alle informazioni ad essi collegate.
- *RecensioneDAO*: gestisce l'accesso ai dati relativi alle recensioni.
- *PreferitoDAO*: gestisce l'accesso ai dati relativi ai ristoranti presenti tra i preferiti degli utenti.

I DAO non adottano necessariamente una nomenclatura CRUD uniforme, ma espongono metodi specifici in funzione delle operazioni richieste dall'applicazione. Tra le operazioni presenti nei DAO si trovano, ad esempio:

- inserimento di nuovi dati;

- ricerca e recupero di dati tramite identificativo o altri criteri;
- aggiornamento dei dati;
- cancellazione dei dati;
- recupero di liste di elementi;
- gestione delle associazioni tra entità.

Ad esempio, *RistoranteDAO* espone operazioni per la ricerca dei ristoranti, il recupero di un ristorante tramite identificativo, l'inserimento e la cancellazione di un ristorante e il calcolo delle relative statistiche. *ServizioDAO* gestisce inoltre la sostituzione delle associazioni tra ristoranti e servizi.

I DAO utilizzano *PreparedStatement* per l'esecuzione delle interrogazioni parametrizzate, separando i dati forniti dall'applicazione dal codice SQL.

Pattern Service

Il pattern **Service** è implementato nel modulo *theknife-server* e parzialmente nel modulo *theknife-common* e ha il compito di raccogliere la logica applicativa e di business, utilizzando i DAO per accedere ai dati persistenti.

Tra le classi *Service* presenti nel modulo si trovano:

- **UtenteService**: gestisce la logica applicativa relativa agli utenti, inclusa la registrazione e l'autenticazione;
- **RistoranteService**: gestisce la logica applicativa relativa ai ristoranti, comprese le operazioni di inserimento, ricerca e gestione delle informazioni dei ristoranti;
- **RecensioneService**: gestisce la logica applicativa relativa alle recensioni.
- **CucinaTranslationService** in *theknife-common*: gestisce la traduzione bidirezionale EN↔IT dei termini di cucina e stile presenti nel dataset, mediante una tabella statica di 380 corrispondenze.

Il livello *Service* costituisce quindi un livello intermedio tra le funzionalità dell'applicazione e i DAO, evitando di inserire la logica applicativa direttamente nelle classi di accesso ai dati.

Persistenza dei dati

La persistenza dei dati è gestita mediante il sistema di gestione di basi di dati **PostgreSQL**, scelto come componente responsabile della memorizzazione persistente delle informazioni e della gestione dei vincoli di integrità e delle transazioni.

Configurazione della connessione alla base di dati

La configurazione della connessione alla base di dati è gestita mediante la classe *ConfigurazioneDB* presente nel package radice *theknife.server*, che fornisce i parametri necessari per stabilire la connessione, tra cui:

- *host*: l'host della base di dati;
- *port*: la porta utilizzata per la connessione (5432, valore predefinito di PostgreSQL);
- *database*: il nome della base di dati (dbTK);
- *username*: il nome dell'utente utilizzato per l'accesso;
- *password*: la password utilizzata per l'accesso.

Gestione delle connessioni

La gestione delle connessioni alla base di dati è affidata alla classe *DatabaseManager* nel modulo *theknife-server*. I DAO richiedono una connessione tramite il metodo *getConnection()* e utilizzano la connessione per eseguire le operazioni SQL.

Le connessioni, così come gli altri oggetti JDBC utilizzati dai DAO, vengono gestite mediante ***try-with-resources***, garantendone la chiusura automatica al termine dell'operazione anche in caso di errore.

Questa scelta permette di evitare la gestione manuale della chiusura delle risorse JDBC e riduce il rischio di lasciare connessioni o altri oggetti aperti.

Gestione delle transazioni

La gestione delle transazioni è realizzata a livello applicativo mediante JDBC, sfruttando il supporto transazionale fornito da PostgreSQL.

La maggior parte dei metodi DAO esegue una singola istruzione SQL utilizzando una nuova connessione e non richiede quindi una gestione esplicita della transazione nel codice. Nei casi in cui un'operazione richiede più istruzioni SQL che devono essere eseguite atomicamente, viene invece utilizzata una transazione esplicita.

Un esempio è rappresentato dal metodo *associa()* di *ServizioDAO*, che sostituisce i servizi associati a un ristorante. Il metodo disabilita l'auto-commit, elimina le associazioni precedenti, inserisce quelle nuove ed esegue il commit solo al termine positivo dell'operazione. In caso di errore viene eseguito il rollback, annullando le modifiche effettuate durante la transazione.



Gestione delle eccezioni

La gestione delle eccezioni relative all'accesso ai dati è implementata nel modulo *theknife-server*. Le *SQLException* generate durante le operazioni JDBC vengono intercettate dai DAO e convertite in *DataAccessException*, evitando di propagare direttamente i dettagli dell'implementazione JDBC ai livelli superiori dell'applicazione.

Nel livello applicativo sono inoltre presenti eccezioni dedicate alla validazione dei dati e alla gestione degli errori applicativi, tra cui:

- *DataAccessException*: rappresenta gli errori verificatisi durante l'accesso alla base di dati;
- *ValidationException*: rappresenta gli errori relativi alla validazione dei dati;
- *ApplicationException*: rappresenta gli errori di carattere applicativo.

6.2 Gestione degli errori

La gestione degli errori è stata progettata per garantire la robustezza e la prevedibilità del comportamento dell'applicazione in presenza di condizioni anomale.

Gli errori sono gestiti mediante eccezioni personalizzate, utilizzate per distinguere le diverse tipologie di errore, e mediante meccanismi di gestione presenti nei *Command*, che traducono le eccezioni in risposte da restituire al client.

Eccezioni personalizzate

Il progetto definisce tre eccezioni personalizzate nel package *theknife.server.exception*: *ApplicationException*, *DataAccessException* e *ValidationException*. Le tre classi **non costituiscono una gerarchia di ereditarietà tra loro**, ma estendono direttamente la classe *Exception*. La distinzione tra le diverse eccezioni permette di classificare gli errori in base alla loro natura e di associarli a differenti modalità di risposta da parte del server.

ApplicationException

La classe *ApplicationException* rappresenta un errore di tipo applicativo o di dominio, dovuto alla violazione di una regola dell'applicazione nonostante i dati forniti siano validi.

Un esempio è rappresentato dal tentativo di effettuare un'operazione non consentita dalle regole di business, come inserire una seconda recensione per lo stesso ristorante. La classe estende direttamente *Exception* e riceve nel costruttore il messaggio descrittivo dell'errore.

DataAccessException

La classe *DataAccessException* rappresenta gli errori verificatisi durante l'accesso alla base di dati. Anche questa classe estende direttamente *Exception* e viene utilizzata dai DAO per impedire che le eccezioni specifiche di JDBC, come le *SQLException*, vengano propagate direttamente ai livelli superiori dell'applicazione. Le *SQLException* vengono infatti intercettate nei DAO e convertite in *DataAccessException*.

Ad esempio, nei DAO un errore durante l'esecuzione di un'operazione SQL viene intercettato e trasformato in una *DataAccessException*, mantenendo così separato il livello di accesso ai dati dai dettagli dell'implementazione JDBC.

ValidationException

La classe *ValidationException* rappresenta gli errori relativi alla validazione dei dati forniti dall'utente. Anche questa classe estende direttamente *Exception* e contiene il messaggio descrittivo dell'errore. Viene utilizzata, ad esempio, quando mancano dati obbligatori oppure quando un valore non rispetta i vincoli previsti dall'applicazione.

La distinzione tra le tre tipologie di eccezione consente quindi di differenziare gli errori imputabili alla richiesta dell'utente, quelli dovuti alle regole applicative e quelli causati da problemi del sistema o della base di dati.

Gestione delle eccezioni nei Command

La gestione delle eccezioni avviene principalmente all'interno delle classi *Command* del modulo *theknife-server*. Il metodo *execute()* dell'interfaccia *Command* non dichiara eccezioni, ma richiede che ogni errore venga intercettato e trasformato in una *Response* contenente il relativo *ResponseStatus*. Questa scelta evita che un'eccezione non gestita interrompa l'esecuzione del thread che sta servendo il client senza restituire una risposta.

I *Command* utilizzano blocchi *try-catch* per intercettare le diverse tipologie di eccezione. In particolare, a seconda dell'operazione, vengono gestite separatamente:

- *ValidationException*, generalmente convertita in *ResponseStatus.ERRORE_VALIDAZIONE*;
- *ApplicationException*, convertita in un *ResponseStatus* coerente con l'errore applicativo verificatosi;
- *DataAccessException*, convertita in *ResponseStatus.ERRORE_SERVER*;
- eventuali altre eccezioni, gestite come errori imprevisti del server.

Ad esempio, nel comando *AccediCommand* un errore di validazione viene restituito come *ERRORE_VALIDAZIONE*, un errore applicativo come *ERRORE*, mentre un errore di accesso alla base di dati o un'eccezione non prevista vengono restituiti come *ERRORE_SERVER*.

Gestione degli errori di accesso ai dati

Gli errori provenienti dal livello di persistenza vengono gestiti dai DAO mediante blocchi *try-catch*. Le *SQLException* generate dalle operazioni JDBC vengono intercettate e convertite in *DataAccessException*, che viene successivamente

propagata al livello applicativo. In questo modo i livelli superiori non devono dipendere direttamente dalle eccezioni specifiche di JDBC.

Gestione degli errori di validazione

Gli errori relativi ai dati in ingresso vengono individuati principalmente nel livello *Service*, dove vengono effettuati i controlli sui dati ricevuti. In caso di violazione dei vincoli previsti, viene sollevata una *ValidationException*, successivamente intercettata dal *Command* e trasformata in una risposta con stato *ERRORE_VALIDAZIONE*.

Ad esempio, il servizio relativo alla ricerca dei ristoranti nelle vicinanze verifica che il luogo sia valorizzato e che il raggio sia maggiore di zero, sollevando una *ValidationException* in caso contrario.

Gestione degli errori applicativi

Gli errori relativi alle regole di business vengono rappresentati mediante *ApplicationException*. Questa eccezione permette di distinguere una condizione applicativa non consentita da un errore tecnico della base di dati o da un errore di validazione.

Un esempio è il tentativo di inserire una recensione già esistente per la stessa combinazione di cliente e ristorante: il DAO intercetta il relativo errore PostgreSQL e lo traduce in *ApplicationException*.

Gestione degli errori lato client

Sul lato client la distinzione tra errore applicativo ed errore di comunicazione è realizzata mediante la classe *ErroreServerException*, definita nel package *theknife.client.service*. La classe segnala che il server ha elaborato la richiesta restituendo una *Response* con stato diverso da SUCCESSO, ad esempio in caso di validazione fallita, credenziali errate o operazione non autorizzata; l'eccezione trasporta il campo *messaggio*, che contiene un testo pensato per la presentazione all'utente finale.

I tre service del client (*AuthService*, *RistoranteService* e *RecensioneService*) verificano lo stato della risposta ricevuta e, quando questo non è SUCCESSO, lanciano una *ErroreServerException* contenente il messaggio restituito dal server. La classe estende *IOException* perché ciò consente di distinguere due situazioni che altrimenti risulterebbero indistinguibili:

- *ErroreServerException*: il server ha risposto e ha segnalato un errore applicativo;



- *IOException* o *ClassNotFoundException* sollevata da *ServerConnection.inviaRichiesta()*: la comunicazione con il server non è andata a buon fine (connessione caduta, server irraggiungibile, deserializzazione fallita) e nessun errore applicativo è stato segnalato.

La distinzione viene utilizzata dalla classe *TaskRunner* del package *ui*, che esegue le chiamate ai service fuori dal *JavaFX Application Thread* e mostra l'esito mediante un *Toast*. In caso di *ErroreServerException* il messaggio mostrato è quello ricevuto dal server, riportato così com'è; per le altre eccezioni di comunicazione viene mostrato un messaggio generico di connessione, evitando di esporre all'utente testi tecnici non significativi.

6.3 Gestione della sicurezza

La gestione della sicurezza è stata progettata per proteggere le credenziali degli utenti, controllare l'accesso alle funzionalità dell'applicazione e mantenere l'associazione tra le richieste e l'utente autenticato.

Nel modulo *theknife-server* la sicurezza è articolata principalmente in tre aspetti: autenticazione degli utenti, autorizzazione delle operazioni e gestione delle sessioni.

Gestione dell'autenticazione

L'autenticazione degli utenti è gestita dal livello *Service* mediante la classe *UtenteService*, che coordina l'accesso ai dati tramite *UtenteDAO* e la gestione delle sessioni tramite *SessionManager*. Il servizio si occupa sia della registrazione degli utenti sia della verifica delle credenziali durante l'accesso.

Registrazione degli utenti

La registrazione è gestita dal metodo *registra()* di *UtenteService*. Prima di procedere con la persistenza vengono verificati i dati obbligatori (nome, cognome, username, email, password, data di nascita e ruolo) e i relativi vincoli: la password deve avere almeno 8 caratteri, l'email deve rispettare il formato previsto e la data di nascita deve corrispondere ad almeno 18 anni compiuti. Vengono inoltre verificate l'unicità dello username e dell'email, che in caso di conflitto producono una *ApplicationException*. I controlli vengono effettuati direttamente nel server e non sono quindi demandati esclusivamente all'interfaccia client.

Il comando *RegistratiCommand* delega la registrazione a *UtenteService* e non richiede che l'utente sia già autenticato. La registrazione non comporta inoltre l'apertura automatica di una sessione: dopo la registrazione l'utente deve effettuare esplicitamente l'accesso tramite l'operazione di autenticazione.

Protezione delle password

Un aspetto rilevante della gestione della sicurezza riguarda la protezione delle password. *UtenteService* utilizza **BCrypt** per ottenere l'hash della password prima che questa venga affidata al DAO per la memorizzazione; il fattore di costo utilizzato è pari a 12. In questo modo la password non viene memorizzata direttamente nella base di dati in forma leggibile.

Durante l'autenticazione, le credenziali fornite dall'utente vengono confrontate con l'hash BCrypt memorizzato nella base di dati. In caso di credenziali non valide viene

utilizzato un messaggio unico per username inesistente e password errata, evitando di rivelare se uno specifico username è presente nel sistema.

UtenteDAO

UtenteDAO costituisce il livello di accesso ai dati degli utenti utilizzato da *UtenteService*. La sua responsabilità è quindi limitata alle operazioni di persistenza e recupero delle informazioni relative agli utenti, mentre la verifica delle credenziali e la logica relativa all'autenticazione rimangono nel livello *Service*.

Gestione delle autorizzazioni

La gestione delle autorizzazioni non è concentrata in *UtenteService* e *UtenteDAO*. Nel progetto il controllo degli accessi è distribuito in base alla responsabilità dell'operazione: il ruolo dell'utente viene verificato dal *Dispatcher*, mentre i controlli relativi alla proprietà delle risorse vengono effettuati nei livelli applicativi interessati.

Il sistema associa infatti all'utente autenticato anche il relativo ruolo, che viene successivamente utilizzato per determinare se l'operazione richiesta è consentita. L'utente della sessione viene passato ai *Command*, consentendo a questi ultimi di effettuare i controlli necessari sull'operazione richiesta.

Controllo della proprietà delle risorse

Oltre al controllo basato sul ruolo, alcune operazioni richiedono di verificare che l'utente autenticato sia effettivamente il proprietario della risorsa sulla quale vuole operare.

Ad esempio, per le operazioni sulle recensioni il controllo che la recensione appartenga all'utente viene effettuato a livello *Service*, prima di delegare al DAO l'operazione di modifica o eliminazione. Il DAO si occupa invece esclusivamente dell'accesso alla base di dati.

Analogamente, per le operazioni relative ai ristoranti viene verificato se l'utente autenticato corrisponde al gestore associato al ristorante.

Un caso concreto è l'eliminazione di un ristorante (comando `ELIMINA_RISTORANTE`): il *Command* rilegge il ristorante tramite il *Service*, confronta il gestore con l'utente di sessione e solo in caso di corrispondenza procede alla cancellazione, che si propaga a cascata su recensioni, servizi e preferiti collegati (vincoli dichiarati nello schema). Trattandosi di un'operazione distruttiva e irreversibile, il controllo di proprietà è qui particolarmente rilevante.



Questa separazione permette di mantenere nei DAO esclusivamente le responsabilità relative alla persistenza, evitando di affidare al livello di accesso ai dati i controlli relativi alle autorizzazioni.

Gestione delle sessioni

La gestione delle sessioni è affidata alla classe *SessionManager*, che mantiene in memoria la corrispondenza tra token di sessione e utente autenticato. Non viene utilizzato un DAO per la gestione delle sessioni: le sessioni sono mantenute direttamente in una *ConcurrentHashMap<String, UtenteDTO>*.

La struttura concorrente permette di gestire correttamente l'accesso alla mappa da parte dei diversi thread associati alle richieste dei client.

Gestione delle sessioni lato client

La gestione della sessione coinvolge anche il modulo *theknife-client*. La classe *AuthService* si occupa di tradurre le operazioni di autenticazione in richieste verso il server. In caso di autenticazione riuscita, il server restituisce un *LoginResultDTO* contenente il token di sessione e le informazioni relative all'utente autenticato; tali informazioni vengono quindi memorizzate localmente tramite la classe *ServerConnection*.

Il token di sessione viene successivamente inserito nelle richieste inviate al server, permettendo al *Dispatcher* di associare ogni richiesta all'utente autenticato corrispondente. In questo modo l'identità dell'utente non viene ricavata dai dati contenuti nel payload delle singole operazioni, ma dalla sessione associata alla richiesta.

L'operazione di logout viene gestita da *AuthService* mediante l'invio del comando *ESCI*. Sul server il token viene rimosso dal *SessionManager*, invalidando la sessione associata.

Apertura della sessione

In seguito a un'autenticazione corretta, *UtenteService* genera un token di sessione, registra la corrispondenza tra token e utente nel *SessionManager* e restituisce il token al client insieme alle informazioni necessarie sull'utente autenticato.

Il token viene quindi utilizzato nelle richieste successive per identificare l'utente associato alla sessione.



Recupero dell'utente dalla sessione

Il metodo *getUtenteFromSession()* del *SessionManager* riceve il token della richiesta e restituisce l'utente associato. Se il token è assente oppure non corrisponde a una sessione presente nella mappa delle sessioni attive, il metodo restituisce NULL.

In questo modo il server può distinguere le richieste effettuate da utenti autenticati da quelle provenienti da client che non dispongono di una sessione valida.

Chiusura della sessione

La chiusura della sessione è gestita tramite il metodo *logout()* del *SessionManager*, che rimuove dalla mappa il token associato alla sessione. Il comando *EsciCommand* delega questa operazione a *UtenteService* e restituisce al client una risposta che conferma la chiusura della sessione.

Persistenza delle sessioni

Le sessioni sono mantenute **esclusivamente in memoria** e non vengono persistite nella base di dati. Di conseguenza, le sessioni attive vengono perse quando il server viene riavviato e gli utenti devono effettuare nuovamente l'autenticazione.

6.4 Gestione dei servizi di geolocalizzazione

La gestione dei servizi di geolocalizzazione si articola in due componenti del modulo *theknife-server*: *GeocodingClient*, che fornisce geocoding diretto e inverso tramite *Nominatim/OpenStreetMap* per la ricerca per località, la ricerca nelle vicinanze e l'inserimento dei ristoranti, e *LocalizzazioneIpClient*, che fornisce una stima approssimata della posizione dell'utente tramite il servizio esterno *ip-api.com*.

Gestione della geocodifica

Accanto al servizio di localizzazione tramite indirizzo IP, il package *theknife.server.external* comprende la classe *GeocodingClient*, che implementa il geocoding diretto e inverso mediante il servizio *Nominatim* di *OpenStreetMap*. Il geocoding diretto traduce un indirizzo testuale nelle coordinate geografiche corrispondenti ed è utilizzato da *RistoranteService* per la ricerca per località, la ricerca nelle vicinanze e l'inserimento di un nuovo ristorante, le cui coordinate vengono calcolate dal server a partire dall'indirizzo. Il geocoding inverso, implementato dal metodo *nomeLuogo()*, consente di tradurre una coppia di coordinate nel nome leggibile del luogo corrispondente.

La classe utilizza due endpoint del servizio:

- <https://nominatim.openstreetmap.org/search> per il geocoding diretto;
- <https://nominatim.openstreetmap.org/reverse> per il geocoding inverso.

L'accesso al servizio è conforme alle condizioni d'uso di *Nominatim*, che prevede l'identificazione del chiamante e la limitazione della frequenza delle richieste. A tale scopo la classe definisce un'intestazione User-Agent esplicita, senza la quale le richieste anonime vengono rifiutate, e un intervallo minimo di 1000 millisecondi fra due richieste consecutive. Il limite di frequenza è applicato a livello di classe: il metodo *attendiTurno()* mantiene un riferimento condiviso all'istante dell'ultima richiesta inviata e sospende il thread chiamante finché non sia trascorso l'intervallo minimo, anche in presenza di più thread client che utilizzano il server contemporaneamente.

Campi e metodi principali

La classe *GeocodingClient* utilizza i seguenti elementi principali:

- *BASE_RICERCA* e *BASE_REVERSE*: gli endpoint del geocoding diretto e inverso;
- *USER_AGENT*: l'intestazione identificativa richiesta dalla policy di *Nominatim*;
- *TIMEOUT*: un timeout di 2 secondi per la connessione e per la singola richiesta;
- *INTERVALLO_MINIMO_MS*: l'intervallo minimo fra due richieste (1000 ms);

- *CAMPO_LAT*, *CAMPO_LON*, *CAMPO_NOME*: espressioni regolari utilizzate per estrarre dalla risposta la latitudine, la longitudine e il nome del luogo (campo *display_name*); quest'ultima tiene conto delle sequenze di escape, frequenti nei nomi di luogo;
- *http*: istanza di *HttpClient* riutilizzata per tutte le richieste.

Tra i principali metodi della classe sono presenti:

- *GeocodingClient()*: costruisce il client HTTP configurandone il timeout di connessione e la gestione dei redirect;
- *geocodifica(String indirizzo)*: traduce un indirizzo completo (via, città, nazione) in un oggetto *PosizioneDTO* contenente coordinate e nome del luogo; restituisce NULL se l'indirizzo è vuoto o assente;
- *nomeLuogo(double latitudine, double longitudine)*: restituisce il nome del luogo corrispondente alle coordinate indicate;
- *chiama(String url)*: esegue la richiesta HTTP rispettando l'intervallo minimo fra le chiamate;
- *attendiTurno()*: applica il limite di frequenza condiviso fra tutti i thread;
- *estraiTesto(Pattern campo, String corpo)* ed *estraiNumero(Pattern campo, String corpo)*: isolano dalla risposta i valori necessari mediante le espressioni regolari definite.

Gestione delle richieste http della geocodifica

Analogamente alla localizzazione tramite IP, la classe utilizza il client HTTP standard del JDK, senza introdurre librerie esterne né un parser JSON dedicato. La richiesta viene costruita specificando l'URI dell'endpoint, l'intestazione User-Agent, l'intestazione Accept: *application/json*, il timeout della richiesta e il metodo GET. Nel geocoding diretto, il termine ricercato viene codificato mediante *URLEncoder* e la ricerca è limitata al primo risultato (parametro *limit=1*); nel geocoding inverso vengono passati le coordinate e un livello di zoom (*zoom=10*) che determina la granularità del luogo restituito.

Gestione della risposta della geocodifica

La risposta JSON viene interpretata estraendo i soli valori necessari mediante le espressioni regolari descritte. Nel geocoding diretto, il metodo verifica la presenza di latitudine e longitudine; quando i dati sono validi li converte in valori numerici e costruisce un oggetto *PosizioneDTO*, utilizzando come nome del luogo il valore del campo *display_name* oppure, in sua assenza, l'indirizzo richiesto. Nel geocoding inverso viene restituito il solo nome del luogo.

Gestione degli errori della geocodifica

Il geocoding costituisce una funzionalità migliorativa e non indispensabile: ogni fallimento è gestito in modo silenzioso e non interrompe l'operazione che ha richiesto la chiamata. Il metodo *chiama()* restituisce NULL in caso di errore di rete, timeout, codice di stato diverso da 200 o thread interrotto; in quest'ultimo caso lo stato di interruzione viene ripristinato mediante *Thread.currentThread().interrupt()*. Anche *geocodifica()* e *nomeLuogo()* restituiscono NULL senza propagare eccezioni al chiamante.

Sul piano applicativo, l'assenza di una posizione stimata viene gestita da *RistoranteService* con un ripiego sulla base di dati: quando il geocoding non produce risultati, le coordinate del luogo vengono richieste a *RistoranteDAO.trovaCoordinateLuogo*. Nella ricerca nelle vicinanze, se anche questo tentativo fallisce, l'operazione termina con una *ApplicationException*; nell'inserimento di un ristorante, invece, il ristorante viene creato comunque, senza coordinate, e l'esito del geocoding viene comunicato al client attraverso il campo dedicato dell'oggetto *EsitoAggiuntaRistorante*, che contiene il *RistoranteDTO* creato e un indicatore sulla validità delle coordinate geografiche restituite dal servizio di geocoding.

Ricerca dei ristoranti nelle vicinanze

Il comando CERCA_VICINO (RF-03 per l'Ospite, RF-07 per il Cliente — raggiungibile da entrambi, non richiede autenticazione) restituisce i ristoranti entro un raggio scelto dall'utente da un luogo indicato (o dal proprio domicilio, se autenticato). *RistoranteService* traduce il nome del luogo in coordinate tramite *GeocodingClient*, con ripiego sulla media delle coordinate dei ristoranti già presenti in quella città se il geocoding non risponde (decisione 17). Le coordinate arrivano già convertite al livello DAO: *RistoranteDAO.cercaVicino* applica soltanto il filtro geografico sul raggio, senza alcuna conversione nome-luogo (decisione 14), calcolando la distanza direttamente in SQL con la formula di Haversine e ordinando i risultati per distanza crescente.

Servizio di geolocalizzazione

La gestione dei servizi di geolocalizzazione è stata progettata per fornire una **stima approssimata della posizione dell'utente**, utilizzata come informazione di supporto per la compilazione della località e per facilitare le funzionalità dell'applicazione che richiedono informazioni geografiche.

La stima della posizione costituisce un'informazione **accessoria e non indispensabile al funzionamento dell'applicazione**: se il servizio esterno non è raggiungibile, non

riconosce l'indirizzo IP o restituisce una risposta non valida, il client di localizzazione restituisce NULL senza propagare l'errore ai livelli superiori. L'utente può quindi continuare a utilizzare l'applicazione inserendo manualmente la località.

Il servizio di geolocalizzazione è implementato mediante la classe *LocalizzazioneIpClient*, appartenente al package *theknife.server.external*. La classe utilizza il client HTTP del JDK e interroga l'endpoint *http://ip-api.com/json/*. La richiesta può contenere l'indirizzo IP del client da geolocalizzare; quando tale indirizzo non è disponibile, viene utilizzata una variante che consente al servizio esterno di geolocalizzare il chiamante, ossia la macchina su cui è in esecuzione il server.

Nel normale funzionamento viene quindi utilizzato **l'indirizzo IP del client**, mentre la localizzazione dell'IP del server costituisce un comportamento di ripiego per richieste che non contengono l'indirizzo del client.

Campi e metodi principali

La classe *LocalizzazioneIpClient* utilizza i seguenti elementi principali:

- *BASE_ENDPOINT*: definisce l'endpoint del servizio esterno;
- *CAMPI*: specifica i soli campi richiesti nella risposta, ovvero *status*, *lat*, *lon* e *city*;
- *TIMEOUT*: definisce un timeout di 2 secondi per la connessione e le richieste;
- *CAMPO_STATO*: espressione regolare utilizzata per estrarre l'esito della richiesta;
- *CAMPO_LAT*: espressione regolare utilizzata per estrarre la latitudine;
- *CAMPO_LON*: espressione regolare utilizzata per estrarre la longitudine;
- *CAMPO_CITTA*: espressione regolare utilizzata per estrarre il nome della città;
- *http*: istanza di *HttpClient* riutilizzata per le richieste.

Tra i principali metodi della classe sono presenti:

- *LocalizzazioneIpClient()*: costruisce il client HTTP configurandone il timeout e la gestione dei redirect;
- *posizioneStimata()*: richiede la stima della posizione della macchina del server come comportamento di ripiego;
- *posizioneStimata(String ip)*: richiede la stima della posizione associata all'indirizzo IP indicato;

- *estrai(Pattern campo, String corpo)*: estrae dal corpo della risposta il primo valore corrispondente all'espressione regolare specificata.

Gestione delle richieste http della localizzazione tramite IP

La gestione delle richieste HTTP è realizzata mediante le classi del client HTTP standard del JDK. Il client viene configurato con un **timeout di 2 secondi** e con la gestione dei redirect tramite `HttpClient.Redirect.NORMAL`. La scelta di un timeout breve è coerente con la natura accessoria della funzionalità: la richiesta di localizzazione non deve ritardare significativamente le operazioni dell'utente.

La richiesta viene costruita mediante *HttpRequest.newBuilder()*, specificando:

- l'URI dell'endpoint;
- l'intestazione *Accept: application/json*;
- il timeout della richiesta;
- il metodo HTTP GET.

La risposta viene quindi ottenuta mediante *http.send()* e acquisita come stringa.

Gestione della risposta della localizzazione tramite IP

La risposta del servizio contiene dati in formato JSON. Nel progetto non viene utilizzato un parser JSON dedicato: i valori necessari vengono estratti mediante espressioni regolari definite nella classe *LocalizzazioneIpClient*. In particolare, vengono estratti lo stato della richiesta, la latitudine, la longitudine e il nome della città.

Prima di costruire la posizione, il metodo verifica che il codice HTTP della risposta sia *200* e che il campo status abbia valore *success*. Viene inoltre verificata la presenza di latitudine e longitudine. In caso contrario, il metodo restituisce `NULL`. Quando i dati sono validi, latitudine e longitudine vengono convertite in valori numerici e utilizzate, insieme al nome della città, per costruire un oggetto *PosizioneDTO*.

Gestione degli errori della localizzazione tramite IP

La gestione degli errori è realizzata mediante blocchi *try-catch*. Le eccezioni che possono verificarsi durante la costruzione o l'esecuzione della richiesta HTTP e durante l'elaborazione della risposta non vengono propagate al chiamante. In particolare, quando viene intercettata una *InterruptedException*, il thread corrente viene nuovamente marcato come interrotto mediante



`Thread.currentThread().interrupt()`, dopodiché il metodo restituisce NULL. Le altre eccezioni vengono intercettate e gestite restituendo anch'esse NULL.

Questo comportamento rende il servizio di geolocalizzazione **non bloccante rispetto alle funzionalità principali dell'applicazione**: l'assenza del servizio esterno, un indirizzo IP non geolocalizzabile o un errore nella comunicazione determinano semplicemente l'assenza della posizione stimata, senza compromettere il completamento dell'operazione richiesta dall'utente.

6.5 Patterns significativi

Nel progetto sono stati adottati diversi design patterns e meccanismi architetturali con l'obiettivo di favorire la separazione delle responsabilità, la modularità e la manutenibilità del codice. Tra i principali si distinguono il pattern *Command*, utilizzato per strutturare la gestione delle operazioni del server, il pattern *DAO* per separare l'accesso ai dati dalla logica applicativa, il pattern *Singleton* e il pattern *Facade* per la gestione della connessione client-server, il principio *Observer* attraverso i meccanismi di osservazione e binding di JavaFX e il pattern *Thread Pool* per la gestione concorrente delle richieste del server.

Pattern Command

Il pattern *Command* è stato adottato nel modulo *theknife-server* e in parte nel modulo *theknife-common* per gestire le operazioni del server in modo strutturato e estensibile. Il pattern *Command* rappresenta ogni operazione come un oggetto autonomo, permettendo di introdurre nuove operazioni attraverso l'aggiunta di nuovi comandi, limitando le modifiche alle componenti già esistenti.

Inoltre, questo pattern facilita la documentazione del codice con UML e la comprensione delle operazioni disponibili.

Esempi concreti dell'utilizzo (a scopo illustrativo) sono:

- l'interfaccia *Command*, comune per tutti gli handler delle operazioni:

```
public interface Command {  
    Response execute (Request request, UtenteDTO utente);  
}
```

- il metodo *execute* nelle classi *Command*:

```
@Override  
public Response execute (Request request, UtenteDTO utente) {  
    // validazione della richiesta  
    // controllo dell'utente  
    // delega al Service  
    ...  
}
```

- l'enumerativo *CommandType* nel package *theknife.common.enums*, che elenca tutte le operazioni disponibili nel sistema:

```
public enum CommandType {  
    CERCA_RISTORANTI, OTTIENI_DETtagli_RISTORANTE, LEGGI_RECENSIONI, CERCA_VICINO,  
    OTTIENI_LOCALITA_INIZIALE, ACCEDI, REGISTRATI, AGGIUNGI_PREFERITO,  
    RIMUOVI_PREFERITO, VEDI_PREFERITI, AGGIUNGI_RECENSIONE, MODIFICA_RECENSIONE,  
    ELIMINA_RECENSIONE, LEGGI_RECENSIONI_PERSONALI, AGGIUNGI_RISTORANTE,  
}
```

```
ELIMINA_RISTORANTE, VEDI_RISTORANTI_GESTITI, LEGGI_RECENSIONI_RISTORANTI_GESTITI,
OTTIENI_STATISTICHE_RISTORANTE, RISPONDI_RECENSIONE, ASSOCIA_RISTORANTE, ESCI
}
```

- la classe *CommandDispatcher*, che legge il *CommandType*, istanzia l'handler corretto e lo esegue:

```
public class CommandDispatcher {
    ...
    public Response dispatch(Request request) {
        // instrada la richiesta al Command che la esegue, dopo il gate di
        // autorizzazione.Command che la esegue, dopo il gate di autorizzazione.
    }
    private boolean accessoConsentito(CommandType comando, UtenteDTO utente) {
        // confronta il requisito di accesso del comando con l'utente della
        // sessione.'utente della sessione.
    }
    private Requisito requisitoDi(CommandType comando) {
        // restituisce il livello di accesso richiesto da un comando.
    }
}
```

Pattern DAO

Il pattern Data Access Object (DAO) è stato adottato nel modulo *theknife-server* per separare l'accesso alla base di dati dalla logica applicativa. Ogni DAO incapsula le operazioni di persistenza relative a una specifica entità e mette a disposizione del livello *Service* metodi dedicati al recupero, inserimento, modifica e cancellazione dei dati.

Ad esempio, *RistoranteDAO* costruisce ed esegue le interrogazioni SQL relative ai ristoranti, utilizzando *PreparedStatement* per la parametrizzazione dei valori e gestendo le risorse JDBC mediante *try-with-resources*.

Il DAO non contiene invece la logica applicativa relativa alle operazioni di dominio, che viene demandata al livello *Service*. Ad esempio, *RistoranteService* effettua i controlli sui parametri ricevuti e successivamente delega il recupero dei dati a *RistoranteDAO*.

Un esempio concreto di utilizzo:

```
String sql = "SELECT ... FROM Recensioni rec "
            + "WHERE rec.id_ristorante = ?";

try (Connection conn = db.getConnection();
     PreparedStatement ps = conn.prepareStatement(sql)) {

    ps.setLong(1, idRistorante);

    try (ResultSet rs = ps.executeQuery()) {
        ...
    }
}
```

```
}  
} catch (SQLException e) {  
    throw new DataAccessException(...);  
}
```

Pattern Singleton

Il pattern *Singleton* è stato adottato nel modulo *theknife-client* (package *network*) per garantire che esista una sola istanza della classe *ServerConnection* per tutta la durata dell'applicazione client; garantisce che la connessione socket verso il server sia unica e condivisa da tutti i service.

In particolare, la classe *ServerConnection* include il metodo statico *getInstance()*, che restituisce l'istanza unica della classe *ServerConnection*:

```
ServerConnection.getInstance()
```

L'utilizzo del pattern evita la creazione di connessioni multiple e consente di inizializzare e condividere un'unica istanza di *ServerConnection* tra i diversi service.

Pattern Facade

Il pattern *Facade* è stato adottato nel modulo *theknife-client* (package *network*) per semplificare l'interazione con il sottosistema complesso della comunicazione di rete, nascondendo i dettagli implementativi (apertura del socket, serializzazione, attesa della risposta, deserializzazione e gestione degli errori di rete).

Il pattern *Facade* include la classe *ServerConnection*, che contiene il metodo *inviaRichiesta(Request)*, che invia una *Request* al server e restituisce la *Response*. La classe espone inoltre il metodo *inviaRichiestaAsync(Request)*, una variante asincrona che incapsula la stessa operazione di comunicazione in un oggetto *Task* di JavaFX, restituendolo al chiamante senza attendere la risposta. L'esecuzione del *Task* su un thread separato consente di ottenere lo stesso comportamento non bloccante della versione sincrona utilizzata dai *Service*, mantenendo la gestione della comunicazione interamente all'interno del *Facade*. Anche questa variante rispetta il vincolo di protocollo secondo cui sul socket esiste una sola richiesta alla volta: è l'esecutore del *Task* a determinare il momento dell'invio, mentre la sequenza invio-risposta rimane quella della versione sincrona.

I *Service* utilizzano *ServerConnection* come punto di accesso alla comunicazione con il server, senza dover gestire direttamente i dettagli della connessione socket e del protocollo.

ServerConnection combina quindi due responsabilità di pattern: viene utilizzata come *Singleton* per condividere la connessione e come *Facade* per nascondere ai *Service* i dettagli della comunicazione di rete.

Pattern Observer

Il pattern *Observer* è stato adottato nel modulo *theknife-client* (package *ui*) per aggiornare la GUI quando i dati cambiano. Il pattern *Observer* garantisce che la GUI rimanga sempre sincronizzata con i dati senza che i controller debbano aggiornare manualmente ogni elemento visivo.

Il progetto sfrutta il principio del pattern *Observer* attraverso i meccanismi di osservazione e binding messi a disposizione da JavaFX. Tali meccanismi realizzano il principio del pattern *Observer*, permettendo ai componenti grafici di reagire automaticamente alle variazioni dei dati osservabili.

I vantaggi che derivano dall'utilizzo sono:

- Sincronizzazione.
- Automaticità: I componenti grafici vengono automaticamente notificati quando il loro valore cambia, senza che i controller debbano aggiornare manualmente ogni elemento visivo.
- Modularità: Il pattern *Observer* facilita la modularità del codice, permettendo di separare la logica di aggiornamento della GUI dalla logica di business.

Pattern Thread Pool

Il pattern *Thread Pool* è stato adottato nel modulo *theknife-server* (package *network*) per gestire concorrentemente le richieste provenienti da più client. Il pattern *Thread Pool* è implementato mediante l'utilizzo di *ExecutorService*, che gestisce l'esecuzione concorrente dei task associati alle richieste dei client e che include il metodo *execute* che sottomette un task al pool.

Nel progetto, il pool è a dimensione fissa ed è configurato con un massimo di 20 thread.

I vantaggi che derivano dall'utilizzo sono:

- Concorrenza.
- Efficienza.
- Protezione: l'uso di un pool a dimensione fissa limita il numero massimo di connessioni gestite contemporaneamente, proteggendo il server dall'esaurimento delle risorse sotto carico elevato.

6.6 Ottimizzazioni delle prestazioni

Le ottimizzazioni delle prestazioni sono state introdotte con l'obiettivo di migliorare la reattività dell'applicazione e la capacità di gestire richieste concorrenti. Tra le principali scelte adottate rientrano l'utilizzo di un thread pool nel server per la gestione concorrente delle richieste e l'esecuzione delle chiamate al server su thread separati rispetto al *JavaFX Application Thread*. A livello di accesso ai dati, vengono inoltre utilizzate transazioni JDBC esplicite nei casi in cui un'operazione richiede più istruzioni SQL che devono essere eseguite atomicamente.

Thread Pool

Il thread pool è utilizzato nel modulo *theknife-server* per gestire concorrentemente le richieste provenienti da più client. Il server utilizza un *ExecutorService* del JDK configurato come pool a dimensione fissa, al quale vengono sottoposti i task associati alle connessioni dei client.

Le richieste vengono sottomesse al pool mediante il metodo *execute(Runnable)*. La configurazione a dimensione fissa limita il numero di task che possono essere eseguiti contemporaneamente, evitando la creazione indiscriminata di nuovi thread.

I vantaggi che ne derivano sono:

- **Concorrenza:** consente di gestire richieste provenienti da client differenti in parallelo.
- **Efficienza:** permette di riutilizzare i thread del pool, evitando la creazione di un nuovo thread per ogni richiesta.
- **Controllo delle risorse:** la dimensione fissa del pool limita il numero di thread utilizzabili contemporaneamente.

Transazioni JDBC

Le transazioni JDBC sono utilizzate nel modulo *theknife-server* nei casi in cui un'operazione richiede l'esecuzione di più istruzioni SQL che devono essere completate atomicamente. In questi casi viene disabilitato l'auto-commit della connessione e le operazioni vengono concluse mediante *commit()* oppure annullate mediante *rollback()* in caso di errore.

La gestione della transazione utilizza i metodi JDBC:

- *setAutoCommit(false)*: disabilita il commit automatico;
- *commit()*: rende definitive le modifiche effettuate dalla transazione;
- *rollback()*: annulla le modifiche effettuate nella transazione in caso di errore.

I vantaggi che ne derivano sono:

- **Atomicità:** le istruzioni che fanno parte della stessa operazione vengono confermate o annullate nel loro insieme.
- **Consistenza:** in caso di errore, il rollback impedisce che rimangano parzialmente applicate le modifiche della transazione.

Chiamate al server su thread separati

Nel modulo theknife-client le chiamate al server sono effettuate mediante due meccanismi complementari.

I controller dell'interfaccia grafica affidano le operazioni di comunicazione con il server che possono richiedere un tempo significativo alla classe *TaskRunner* del package *ui*, che esegue ciascuna chiamata su un executor dedicato e consegna il risultato o l'errore al *JavaFX Application Thread*, dove è sicuro aggiornare i nodi della schermata. Questa scelta evita che l'attesa della risposta dal server blocchi il thread responsabile dell'aggiornamento dell'interfaccia grafica, mantenendo la GUI reattiva. L'executor di *TaskRunner* è a thread singolo per scelta progettuale: poiché *ServerConnection* (che implementa la comunicazione con il server e fornisce il metodo *inviaRichiesta(Request)* per l'invio delle richieste e la ricezione delle relative risposte) possiede un solo socket e un solo stream di scrittura, due richieste contemporanee si intreccerebbero, facendo leggere a ciascuna la risposta dell'altra; l'esecuzione sequenziale garantisce quindi che le richieste siano inviate una alla volta e nell'ordine di arrivo.

In alternativa ai controller, *ServerConnection* mette a disposizione direttamente il metodo *inviaRichiestaAsync(Request)*, che restituisce un oggetto *Task* di JavaFX incapsulando la comunicazione; il chiamante può eseguirlo su qualunque thread separato e riceverne gli eventi di completamento sull'*Application Thread*, secondo il modello standard di concorrenza di JavaFX.

I vantaggi che ne derivano sono:

- **Reattività:** il *JavaFX Application Thread* rimane libero di gestire gli eventi e l'aggiornamento della GUI durante l'attesa delle risposte del server.
- **Separazione delle responsabilità:** la gestione della comunicazione di rete resta in *ServerConnection*, mentre la schedulazione delle operazioni compete a *TaskRunner*.
- **Correttezza del protocollo:** l'accesso al socket condiviso avviene in modo sequenziale, evitando l'intreccio di richieste e risposte.

7. Installazione ed esecuzione

Il progetto *TheKnife* è un sistema client-server per la ricerca e la recensione di ristoranti. Questa sezione descrive i prerequisiti software, la configurazione della base di dati, la compilazione e le modalità di avvio del server e del client.

Prerequisiti

Per compilare ed eseguire il progetto sono necessari:

- **JDK 21**: il progetto è sviluppato utilizzando Java 21;
- **PostgreSQL**: utilizzato come sistema di gestione della base di dati;
- **Maven**: utilizzato per la compilazione e la costruzione del progetto, organizzato come applicazione multi-modulo Maven.

Configurazione della base di dati

Prima di avviare il server è necessario predisporre una base di dati PostgreSQL.

Il modo più semplice è tramite Maven, dalla directory radice del progetto:

```
mvn initialize -Pdb-setup -Ddb.admin.user=postgres -Ddb.admin.password=miapassword
```

Il comando crea il ruolo applicativo `tk_app`, il database `dbTK`, le tabelle e carica il dataset del docente, eseguendo nell'ordine gli script `Role.sql`, `Schema.sql` e `Data.sql`, descritti sotto. `postgres` e `miapassword` vanno sostituiti con utente e password reali di un amministratore PostgreSQL (i valori dell'esempio non vanno copiati così come sono). Se PostgreSQL non è in ascolto su `localhost:5432`, aggiungere al comando `-Ddb.host=NOMEHOST` e/o `-Ddb.port=NUMEROPORTA`, sempre con i valori reali al posto dei segnaposto.

In alternativa, gli stessi script sono eseguibili a mano con `psql`, dalla cartella `Progettazione/Database/`, nel seguente ordine (`Role.sql` su un database di manutenzione come “`postgres`”, `Schema.sql` e `Data.sql` sul database `dbTK` appena creato):

- *Role.sql*: crea il ruolo applicativo `tk_app` e il database `dbTK`;
- *Schema.sql*: crea le tabelle e i relativi vincoli;
- *Data.sql*: inserisce i dati iniziali necessari all'applicazione.

La configurazione relativa all'accesso alla base di dati viene successivamente richiesta dal server durante l'avvio.

Gli script facoltativi di verifica sono descritti nel capitolo “[Dataset di prova](#)”.

Compilazione del progetto

Il progetto può essere compilato dalla directory radice mediante il comando:

```
mvn clean package
```

Il comando esegue la compilazione dei moduli del progetto e produce i JAR eseguibili nelle rispettive directory target/:

```
theknife-server/target/serverTK.jar  
theknife-client/target/clientTK.jar
```

I JAR prodotti sono autosufficienti e comprendono il modulo *theknife-common* e le dipendenze necessarie all'esecuzione.

Generazione della documentazione JavaDoc

La documentazione JavaDoc di tutti e tre i moduli può essere generata, dalla directory radice, mediante:

```
mvn javadoc:aggregate
```

Il comando produce la documentazione HTML aggregata in target/site/apidocs/, poi copiata nella cartella doc/javadoc/ della consegna.

Esecuzione del server

Il server viene avviato dalla directory radice mediante:

```
java -jar theknife-server/target/serverTK.jar [porta]
```

La porta di ascolto è un parametro opzionale. Se non viene specificata, viene utilizzata la porta predefinita 9999.

Durante l'avvio, il server richiede i parametri necessari per la connessione alla base di dati, ossia *host*, *username* e *password* (rispettivamente localhost, tk_app e TheKnife-B per l'utente applicativo creato dal profilo Maven db-setup).

Esecuzione del client

Dopo l'avvio del server, il client può essere eseguito mediante:

```
java -jar theknife-client/target/clientTK.jar [host] [porta]
```



Anche in questo caso i parametri sono opzionali. Se non vengono specificati, il client utilizza come configurazione predefinita *localhost:9999*.

Il client utilizza quindi *localhost* come host del server e la porta *9999*.

Configurazione dell'host e della porta

L'host e la porta predefiniti utilizzati per la comunicazione client-server sono definiti nel modulo *theknife-common*, nella classe:

```
theknife.common.config.ConfigurazioneServer
```

La presenza di una configurazione condivisa consente di mantenere centralizzati i parametri di connessione utilizzati dal client e dal server.

8. Dataset di prova

Il progetto TheKnife include un dataset di prova predisposto per consentire la verifica delle principali funzionalità dell'applicazione senza dover inserire manualmente tutti i dati necessari. Il dataset comprende un insieme di ristoranti, derivato dalla guida Michelin, e un insieme di utenti di test con ruoli differenti.

Dataset dei ristoranti

Il dataset principale è contenuto nello script *Data.sql* e comprende circa 17000 ristoranti. Per ciascun ristorante sono disponibili le informazioni utilizzate dalle funzionalità di ricerca e visualizzazione, tra cui nome, località, indirizzo, coordinate geografiche, fascia di prezzo, tipologia di cucina e servizi.

Il dataset costituisce quindi la base per verificare funzionalità quali la ricerca dei ristoranti, il filtraggio, la ricerca nelle vicinanze, la visualizzazione dei dettagli e il calcolo delle statistiche relative alle recensioni.

In particolare, il dataset contiene anche i dati necessari alla gestione delle associazioni tra ristoranti e servizi. Nel codice tali associazioni sono rappresentate attraverso la tabella *RistoranteServizio*; i servizi vengono recuperati separatamente e associati ai relativi ristoranti.

Script di verifica opzionali

Oltre agli script *Role.sql*, *Schema.sql* e *Data.sql*, nella cartella Progettazione/Database/ sono presenti tre script facoltativi di supporto alla verifica delle funzionalità, eseguibili in un colpo solo con Maven, dopo che `mvn initialize -Pdb-setup` ha già creato dbTK:

```
mvn initialize -Pdb-test-data
```

Il comando si collega come utente applicativo `tk_app` (già creato dal profilo `db-setup`, nessun argomento richiesto) ed esegue, nell'ordine, questi tre script:

- *DataUtentiTest.sql*: inserisce gli utenti di prova (da eseguire dopo *Schema.sql* e *Data.sql*);
- *AbilitaServiziTest.sql*: abilita prenotazione online e consegna a domicilio su un sottoinsieme distribuito dei ristoranti (da eseguire dopo *Data.sql*);
- *PopolaRecensioniTest.sql*: inserisce recensioni e risposte di prova (da eseguire dopo *DataUtentiTest.sql*, perché le recensioni fanno riferimento agli utenti di prova).



In alternativa, gli stessi tre script sono eseguibili a mano con psql, nello stesso ordine, sul database dbTK.

La cartella contiene infine *Queries.sql*, una raccolta documentativa delle interrogazioni SQL utilizzate dai DAO del server: non è destinata all'esecuzione.

Utenti di prova

Lo script *DataUtentiTest.sql* popola la tabella *Utenti* fornendo quattro utenti di prova, due con ruolo CLIENTE e due con ruolo RISTORATORE, per consentire l'accesso immediato all'applicazione:

Username	Nome	Cognome	Ruolo
<i>cliente_test</i>	Mario	Rossi	CLIENTE
<i>cliente_test2</i>	Giulia	Verdi	CLIENTE
<i>gestore_test</i>	Luca	Bianchi	RISTORATORE
<i>gestore_test2</i>	Anna	Neri	RISTORATORE

Le credenziali sono definite nello script *DataUtentiTest.sql*. La password di tutti gli utenti di prova è *password*; nella base di dati non viene memorizzata in chiaro, ma sotto forma di **hash BCrypt con fattore di costo 12**.

Lo script deve essere eseguito dopo *Schema.sql* e *Data.sql*, in modo da aggiungere gli utenti alle tabelle già create e popolate.

Recensioni di prova

Per consentire la verifica immediata delle funzionalità relative alle recensioni, lo script *PopolaRecensioniTest.sql* popola la tabella *Recensioni* con recensioni fittizie distribuite su circa un quinto dei ristoranti del dataset principale. Lo script deve essere eseguito dopo *Schema.sql*, *Data.sql* e *DataUtentiTest.sql*, in quanto le recensioni fanno riferimento sia ai ristoranti già presenti sia agli utenti di prova con ruolo CLIENTE.

Le recensioni sono inserite mediante query INSERT...SELECT basate sulla funzione CROSS JOIN tra la tabella *RistorantiTheKnife* e l'utente di prova selezionato per username; la distribuzione e il contenuto sono determinati dall'identificativo del ristorante, secondo lo schema seguente:

Inserimento	Username	Ristoranti coinvolti
Prima recensione	<i>cliente_test</i>	<i>id_ristorante</i> divisibile per 5
Seconda recensione	<i>cliente_test2</i>	<i>id_ristorante</i> divisibile per 10
Risposta del gestore		Recensioni con <i>id_ristorante</i> divisibile per 20

In particolare:

- Gli utenti *cliente_test* e *cliente_test2* sono diversi, per cui il vincolo *UNIQUE* (*id_cliente*, *id_ristorante*) non viene violato e ciò consente anche di verificare correttamente la media delle valutazioni e delle statistiche nei ristoranti con più di una recensione.
- La risposta del gestore consente di verificare la corretta visualizzazione delle risposte nella UI e la gestione della risposta da parte dei ristoratori (comando *RISPONDI_RECENSIONE*).
- Anche la valutazione in stelle e la data di pubblicazione sono determinate in modo deterministico dall'identificativo del ristorante.

Dati per la verifica delle funzionalità

Il dataset è predisposto anche per consentire la verifica delle funzionalità che dipendono dalla presenza di specifici attributi dei ristoranti. In particolare, lo script aggiuntivo *AbilitaServiziTest.sql*, relativo ai servizi di prenotazione e consegna, imposta:

- *prenotazione_online* = TRUE per i ristoranti il cui *id_ristorante* è divisibile per 5;
- *consegna_a_domicilio* = TRUE per i ristoranti il cui *id_ristorante* è divisibile per 7.

Il dataset Michelin originale ha tutti i campi impostati a FALSE, per cui i filtri non restituirebbero mai risultati.

Tramite questo script i due filtri possono essere testati su un sottoinsieme distribuito dei ristoranti del dataset.



Relazioni e dati creati durante le prove

Il dataset iniziale fornisce principalmente la base di dati necessaria all'esecuzione dell'applicazione. Alcune funzionalità possono invece essere verificate creando o modificando dati durante l'utilizzo del sistema.

Ad esempio, per testare le funzionalità riservate ai ristoratori è possibile associare un ristorante a uno degli utenti di test con ruolo RISTORATORE. Analogamente, le funzionalità relative alle recensioni e ai preferiti possono essere verificate utilizzando gli utenti con ruolo CLIENTE.



9. Considerazioni progettuali

Organizzazione del progetto in moduli Maven

Il progetto TheKnife è organizzato come un progetto Maven multi-modulo composto da tre moduli: *theknife-common*, *theknife-server* e *theknife-client*. L'organizzazione del progetto in moduli Maven permette di separare le responsabilità e di migliorare la manutenibilità del codice.

Convenzione dei package

Tutti i package del progetto hanno radice *theknife*. (obbligo DIRETTIVE §3.3: il package *theknife* deve esistere e contenere le classi). La convenzione dei package permette di organizzare il codice in modo coerente e di migliorare la leggibilità del codice.

Contenuto del modulo *theknife-common*

Il modulo *theknife-common* contiene esclusivamente le classi condivise tra client e server. Tutte implementano *Serializable* perché viaggiano sul socket tramite *ObjectOutputStream/ObjectInputStream*. Il contenuto del modulo common permette di definire il contratto di comunicazione tra i due JAR e di evitare duplicazione di codice.

Considerazioni sulle sessioni

Il server gestisce le sessioni utente tramite token opachi generati al momento del login. La gestione delle sessioni permette di autenticare gli utenti e di garantire la sicurezza delle operazioni di dominio.

Architettura a strati

L'architettura a strati permette di separare le responsabilità e di migliorare la manutenibilità del codice.

Considerazioni sui design patterns

I design patterns adottati includono il pattern *Command*, il pattern *Singleton*, il pattern *Facade*, il pattern *Observer* e il pattern *Thread Pool*. I design pattern adottati permettono di risolvere problemi specifici e di migliorare la struttura del codice.



Considerazioni sull'ottimizzazione delle prestazioni

Le ottimizzazioni delle prestazioni includono l'uso di un thread pool per la gestione delle connessioni, l'esecuzione delle operazioni di scrittura in una singola transazione JDBC e l'esecuzione delle chiamate al server su thread separati rispetto al *JavaFX Application Thread*. Le ottimizzazioni delle prestazioni permettono di migliorare l'efficienza e la scalabilità del sistema.

10. Limiti della soluzione sviluppata

I principali limiti della soluzione sviluppata riguardano la gestione delle eccezioni, degli errori di rete, delle connessioni concorrenti, delle sessioni, delle transazioni, delle traduzioni, del collaudo e della visualizzazione geografica.

Limiti nella gestione delle eccezioni

La gestione delle eccezioni è stata organizzata mediante eccezioni personalizzate e meccanismi di gestione centralizzata delle risposte di errore.

Una possibile evoluzione consiste nell'introdurre meccanismi ulteriormente centralizzati per la gestione di categorie di errore più ampie e per una distinzione più uniforme tra errori tecnici e errori restituiti all'utente.

Limiti nella gestione degli errori di rete

La gestione degli errori di rete prevede la gestione delle eccezioni generate durante la comunicazione client-server e la restituzione di risposte coerenti all'applicazione client (meccanismo gestito da *ErroreServerException* e *TaskRunner* nel client).

Un possibile miglioramento consiste nell'introdurre meccanismi più evoluti di gestione delle interruzioni della connessione, dei timeout e dei tentativi di riconnessione automatica, attualmente non previsti.

Limiti nella gestione delle connessioni concorrenti

Il server utilizza un thread pool a dimensione fissa per limitare il numero di attività eseguite contemporaneamente. Questa scelta consente di evitare una crescita incontrollata del numero di thread sotto carico elevato, ma introduce un limite alla capacità di gestione simultanea delle richieste: oltre la capacità del pool, le richieste devono attendere che si liberi un thread.

Una possibile evoluzione consiste nell'adottare una strategia di dimensionamento dinamico o una gestione più sofisticata della coda delle richieste, calibrata sul carico previsto.

Limiti sulle sessioni

Le sessioni utente sono mantenute esclusivamente in memoria dal server. Di conseguenza, il riavvio del server comporta la perdita delle sessioni attive e richiede una nuova autenticazione da parte degli utenti. Inoltre, in presenza di più istanze del

server, la gestione delle sessioni richiederebbe un meccanismo di condivisione dello stato o un archivio esterno.

La soluzione adottata è quindi adeguata a un'architettura con una singola istanza del server, ma limita la possibilità di scalare orizzontalmente il componente server senza introdurre ulteriori meccanismi di gestione delle sessioni.

Limiti nella gestione delle transazioni

La gestione delle transazioni è realizzata direttamente mediante JDBC e consente di garantire l'atomicità delle operazioni che coinvolgono più istruzioni SQL. Questa soluzione richiede tuttavia una gestione esplicita di *autoCommit*, *commit* e *rollback* e può diventare più complessa all'aumentare del numero di operazioni transazionali.

In un sistema di dimensioni maggiori potrebbe essere valutato l'utilizzo di un framework di persistenza o di gestione delle transazioni, che permetta di centralizzare e semplificare tali operazioni.

Limiti nella gestione delle traduzioni

Un limite della soluzione riguarda la gestione delle traduzioni utilizzate nella ricerca dei ristoranti. Il dataset di partenza contiene infatti termini relativi alla cucina e allo stile dei ristoranti in lingua inglese, mentre l'applicazione deve consentire la ricerca anche utilizzando i corrispondenti termini italiani.

Per ottenere un comportamento coerente indipendentemente dalla lingua utilizzata nella base di dati e dalla lingua dell'input dell'utente, sarebbe stato possibile ricorrere a un servizio esterno di traduzione. Tale soluzione non è stata adottata a causa dei costi associati ai servizi di traduzione esterni.

È stata quindi realizzata una **tabella di traduzione statica EN↔IT**, contenente i 380 termini di cucina e stile presenti nel dataset. La gestione della ricerca è stata conseguentemente adattata affinché *RistoranteService.cercaRistoranti* utilizzi tale tabella nella normalizzazione e traduzione dei termini di ricerca.

Questa soluzione permette di rendere la ricerca coerente rispetto alle due lingue previste, ma costituisce un limite in termini di estensibilità: l'aggiunta di nuovi termini non presenti nella tabella richiede un aggiornamento manuale della corrispondenza. Un servizio di traduzione esterno consentirebbe invece una gestione più dinamica, a fronte però di costi e di una dipendenza da un servizio di terze parti.

Limiti nel collaudo del sistema

Il progetto non comprende una raccolta di test automatizzati: i moduli non contengono codice di test e la verifica del corretto funzionamento del sistema è stata condotta manualmente, avvalendosi del dataset di prova descritto in precedenza. Questa scelta ha consentito di concentrare le risorse di sviluppo sull'implementazione delle funzionalità, ma comporta una verifica non ripetibile in modo automatico: le regressioni eventualmente introdotte da modifiche successive devono essere individuate ripetendo manualmente le prove, con costi crescenti all'aumentare delle dimensioni del sistema.

Una possibile evoluzione consiste nell'introduzione di test unitari per i componenti a maggiore isolamento, come i *Service* del server, i DTO e i client dei servizi esterni (ad esempio mediante simulazione delle risposte HTTP), e di test di integrazione per la catena *Command* - *Service* - DAO su una base di dati dedicata al collaudo. L'architettura a strati adottata facilita questo sviluppo, in quanto i confini tra i livelli identificano naturalmente le unità verificabili singolarmente.

Limiti nella gestione della mappa geografica

Un ulteriore limite riguarda la visualizzazione geografica dei ristoranti. La soluzione inizialmente adottata prevedeva una mappa geografica interattiva realizzata lato client mediante una webmap basata su JavaScript. Durante le prove, tuttavia, il caricamento della mappa ha mostrato tempi e comportamenti non uniformi, rendendo la visualizzazione meno prevedibile e meno reattiva nell'esperienza d'uso.

Per superare tali problematiche, la soluzione è stata successivamente rivista sostituendo la precedente implementazione basata su *WebView* e JavaScript con un componente nativo JavaFX. La mappa viene costruita utilizzando le immagini dei *tile* di OpenStreetMap, caricate come oggetti *Image* e visualizzate tramite *ImageView* all'interno di un *Pane*. I marker dei ristoranti sono invece rappresentati mediante elementi grafici JavaFX, come *Circle*, mentre le operazioni di spostamento, ingrandimento e rimpicciolimento vengono gestite direttamente in Java attraverso il motore grafico nativo di JavaFX.

Per il caricamento dei *tile* viene inoltre specificato un *User-Agent* esplicito, in conformità alle modalità di accesso previste dal servizio OpenStreetMap.

La nuova implementazione è stata adottata in tutte le schermate dell'applicazione che prevedono la visualizzazione geografica dei ristoranti, in particolare **Risultati**, **Preferiti**, **Dashboard e Dettaglio**. Contestualmente sono stati rimossi i componenti relativi alla precedente soluzione basata su *WebView* e Leaflet, inclusi *mappa.html*, *leaflet.js*,

leaflet.css; il precedente *MappaController* è stato invece riscritto nella versione nativa JavaFX descritta sopra, evitando di mantenere nel progetto codice non più utilizzato.

La nuova soluzione riduce la dipendenza da componenti web e dal bridge tra Java e JavaScript e rende la gestione della mappa maggiormente integrata con l'interfaccia JavaFX. Rimane tuttavia un limite legato alla dipendenza dai *tile* di OpenStreetMap: la visualizzazione della mappa richiede il loro reperimento durante l'esecuzione e può quindi essere influenzata dalla disponibilità e dai tempi di risposta del servizio esterno. Inoltre, l'implementazione nativa richiede una gestione diretta in Java delle operazioni di caricamento, posizionamento dei *tile*, trasformazione delle coordinate e interazione con la mappa, aumentando la complessità del codice rispetto all'utilizzo di una libreria web specializzata.

Un'ulteriore limitazione riguarda la geocodifica: l'accesso al servizio, essendo conforme alle condizioni d'uso di *Nominatim*, prevede l'identificazione del chiamante e la limitazione della frequenza delle richieste. Le richieste anonime vengono quindi rifiutate ed è definito un intervallo minimo di 1000 millisecondi fra due richieste consecutive.



11. Bibliografia e sitografia

- **PostgreSQL**, documentazione ufficiale, Online: <https://www.postgresql.org/>
- **Oracle Java**, documentazione e informazioni sul JDK, Online: <https://www.oracle.com/java/>
- **Apache Maven**, documentazione ufficiale, Online: <https://maven.apache.org/>
- **OpenJFX**, documentazione ufficiale di JavaFX, Online: <https://openjfx.io/>
- **Nominatim / OpenStreetMap**, Online: <https://nominatim.openstreetmap.org/>
- **IP-API**, servizio di geolocalizzazione tramite indirizzo IP, Online: <https://ip-api.com/>
- **BCrypt**, libreria Java utilizzata per l'hashing delle password, Online: <https://github.com/patrickfav/bcrypt>
- **TheFork**, piattaforma di riferimento per la ricerca di ristoranti, Online: <https://www.thefork.it/>
- **Guida Michelin**, dataset dei ristoranti utilizzato nel progetto, Online: <https://www.kaggle.com/datasets/ngshiheng/michelin-guide-restaurants-2021>
- **Oracle Java SE**, documentazione delle API Java, Online: <https://docs.oracle.com/en/java/javase/21/>